A PROJECT REPORT

On

SYNAPSE – College Management Information System (MIS)

Submitted by

Yatin Anil Anchan

in partial fulfillment for the award of the degree

of

BACHELOR OF SCIENCE

in

COMPUTER SCIENCE

under the guidance of

Mr. Hasan Phudinawala

Department of Computer Science

Royal College of Arts, Science and Commerce

(Autonomous)

Semester VI

2025-2026

PORFORMA FOR THE APPROVAL PROJECT PROPOSAL

PNR No.: ..... **Roll no:** _____

1. **Name of the Student**

_____

2. **Title of the Project**

_____

3. **Name of the Guide**

_____

4. **Teaching experience of the Guide** _____

5. **Is this your first submission?** Yes ☐ No ☐

Signature of the Student **Signature of the Guide**

Date: ..... **Date: .....**

Signature of the Coordinator

Date: .....

Since 1989

Royal Higher Education Society's

ROYAL COLLEGE
OF ARTS, SCIENCE & COMMERCE

Empowerment through Value Education

(A Minority Autonomous Institution)

Devoted to Serve

Royal College of Arts, Science and Commerce
BhaktiVedanta Swami Marg, Shrishti Complex, Mira Road East,
Maharashtra 401107

DEPARTMENT OF COMPUTER SCIENCE

CERTIFICATE

This is to certify that **Mr. Yatin Anchan** of **T.Y.B. Sc. Comp Sci (Semester - VI)** class have satisfactorily completed the Project **SYNAPSE – College Management Information System (MIS)**, to be submitted in the partial fulfillment for the award of **Bachelor of Science in Computer Science** during the academic year **2025-2026.**

Date of Submission:

Project Guide **Coordinator**
Computer Science Department

College Seal **Signature of Examiner**

DECLARATION

I, Mr. YATIN ANCHAN, hereby declare that the project entitled "**SYNAPSE – College Management Information System (MIS)**" submitted in partial fulfillment for the award of **Bachelor of Science** in **Computer Science** during the academic year 2025-2026 is my original work and the project has not formed the basis for the award of any degree, associateship, fellowship or any other similar titles.

Signature of the student:

Place:

Date:

ACKNOWLEDGEMENT

Achievement is finding out what you would be doing rather than what you have to do. It is not until you undertake such a project that you realize how much effort and hard work it really is, what your capabilities are and how well you can present yourself or other things. It tells us how much we rely on the efforts and goodwill of others. It gives me immense pleasure to present this report to fulfill my project.

It has been rightly said that we are built on the shoulders of others. For everything I have achieved, the credit goes to all those who helped me complete this project successfully.

I take this opportunity to express my profound to Founder Sir **Prof. Asgar E. Lakdawala**, Principal, **Dr. Kalpana Patankar Jain** and management of **Royal College of Arts, Science & Commerce, Mira Road (E)** for giving me this opportunity to accomplish this project work.

A special vote of thanks to **Prof. Ritika Lala Coordinator of Computer Science Department**, **Mr. Hasan Phudinawala** who is my faculty & my project guide for their most sincere, useful and encouraging contribution throughout the project span.

Finally, I would like to thank the entire Computer Science department who directly or indirectly helped me in the completion of this project & my family without their support, motivation & encouragement this would not have been possible.

YATIN ANCHAN

TABLE OF CONTENTS

Index			
Chapter		**Topic**	**Page No.**
1		**Introduction**	1
	1.1	Background and Project Overview	2
	1.2	Objectives	3
	1.3	Purpose & Scope	4
	1.4	Phase Title	6
	1.5	Gantt Chart	8
2		**System Analysis**	
	2.1	Existing System	9
	2.2	Proposed System	10
	2.3	Requirement Analysis	19
3		**System Design**	
	3.1	UML Diagrams	20
4		**Implementation and Testing**	
	4.1	Code	26
	4.2	Testing Approach	47
5		**Results**	49
6		**Conclusion And Future work**	57
7		**References and Plagiarism Report**	
	7.1	References	59
	7.2	Plagiarism Report	60
	7.3	Approved Project Proposal	63

LIST OF FIGURES

Sr. No.	Names of Figures	Pg. No.
1	Use Case diagram	20
2	Activity diagram	21
3	Sequence diagram	22
4	Class diagram	23
5	Entity Relation Diagram	24
6	Data Flow Diagram	25

CHAPTER 1

INTRODUCTION

1. Introduction

In today's rapidly evolving educational environment, colleges and academic institutions require efficient, centralized and secure systems to manage academic and administrative operations. Traditional manual processes and fragmented digital tools often lead to inefficiencies, data redundancy, lack of transparency and increased administrative workload. To address these challenges, **Management Information Systems (MIS)** play a vital role in streamlining institutional workflows and enhancing decision-making processes.

SYNAPSE is a comprehensive and ultra-niche **Management and Information System (MIS)** designed specifically for colleges to support institutional management, faculty coordination and administrative staff operations. The system aims to provide a unified digital platform that integrates academic, examination, departmental and event management functionalities under a single secure framework.

SYNAPSE follows a **role-based access control model**, ensuring that users such as System Administrators, Principals, Vice Principals (Coordinators of Examination), Heads of Departments (HODs), Faculty Members and Committee Heads can access features relevant to their responsibilities. This structured access enhances data security, accountability and operational efficiency.

The system facilitates end-to-end academic processes including **course, semester and class creation**, examination scheduling and **result generation**. Additional modules such as **attendance management**, **department and committee management** and **event planning with RSVP and guest tracking** further extend the system's capabilities. By centralizing these operations, SYNAPSE minimizes manual effort, reduces errors and improves institutional coordination.

Overall, SYNAPSE is designed to modernize college administration by offering a scalable, secure and efficient MIS solution tailored to the specific needs of academic institutions.

1.1 Background and Project Overview

In the modern academic landscape, colleges face mounting pressures arising from rapidly increasing student enrollments, multifaceted administrative responsibilities, and the growing expectation for real-time, data-driven decision-making. Traditional manual record-keeping methods and fragmented digital tools continue to dominate many institutions, leading to operational bottlenecks, data redundancy, and a high likelihood of human error. Critical processes such as attendance management, examination scheduling, event coordination, and departmental reporting often suffer from delays and inconsistencies, placing an additional burden on administrative staff and limiting effective collaboration among faculty members.

To address these challenges, **SYNAPSE** emerges as a comprehensive and purpose-built solution rooted in established Management Information System (MIS) principles. Rather than functioning as isolated modules, SYNAPSE integrates core academic and administrative operations into a unified platform. The system centralizes academic structure management, user and department coordination, attendance tracking, and secure role-based access control, ensuring that each stakeholder ranging from Principals and Heads of Departments (HODs) to faculty members and staff interacts with data relevant to their responsibilities. This role-aware design enhances accountability, reduces information overload, and ensures data integrity across all levels of the institution.

More than a conventional software application, SYNAPSE is envisioned as a strategic digital backbone for educational institutions. By transforming fragmented workflows into streamlined, automated, and insight-driven processes, the system enables faster decision-making, improved transparency, and operational scalability. As institutions continue to evolve in response to technological advancements and administrative complexity, SYNAPSE lays a robust foundation for future enhancements, supporting sustainable growth and modern governance in higher education.

1.2 Objectives

The primary objectives of SYNAPSE MIS guide its development as a practical solution for college administration challenges. These goals ensure the system delivers efficiency, security and scalability tailored to academic institutions.

1. Design a centralized platform to manage academic and administrative tasks, reducing manual efforts and data mismanagement across departments.
2. Implement role-based access control for users like admins, principals, HODs, faculty and committee heads, ensuring secure and relevant feature access.
3. Automate core academic processes, including course, semester, class creation, faculty assignments and structured record-keeping.
4. Streamline end-to-end examination management, from scheduling and assessments to result generation and reporting.
5. Enable digital attendance tracking for students and faculty, with centralized storage accessible to authorized personnel.
6. Support department and committee oversight, including member assignments, roles and activity logging for better coordination.
7. Enhance transparency and decision-making through real-time data access and organized reporting tools.
8. Build a modular, scalable architecture ready for future additions like analytics, mobile support or integrations.
9. Prioritize user-friendly design and affordability, making advanced MIS features viable for typical college setups.

These objectives directly address inefficiencies in traditional systems, positioning SYNAPSE as a transformative tool for modern education management.

1.3 Purpose and Scope

1.3.1 Purpose

SYNAPSE is developed with a focused purpose to deliver an affordable, secure and centralized Management Information System (MIS) that revolutionizes college administration by tackling inefficiencies in manual and fragmented process and data handling. In today's academic world, institution struggle with outdated record-keeping, disjointed tools and limited access to real-time data, leading to errors and delay and poor coordination.

The system detects and resolves these operational gaps by providing role-based access, automating academic workflows like course management and exams also enabling seamless handling of attendance, departments, classes and students. Upon streamlining these core functions, SYNAPSE generates structured data, enhances, transparency and notifies authorized users instantly preventing administrative bottlenecks before they escalate.

By removing the need for multiple disparate systems and prioritizing intuitive design, SYNAPSE makes advanced institutional management accessible to colleges of all sizes, fostering efficiency, accountability and smarter decision-making across the board.

1.3.2 Scope

SYNAPSE MIS is a native Android application tailored for college administration, providing a lightweight, mobile-first platform that works seamlessly on standard Android devices without needing extra hardware or web dependencies. It's designed to be intuitive and accessible for users like admins, principals, HODs, faculties, running directly on their phones or tablets to handle daily academic and operational tasks efficiently.

The app launches core modules upon login with role-based access, continuously managing workflows like course creation and exam scheduling. It monitors activities such as attendance marking, department updates in real-time, logging data locally and alerting relevant users to issues before they escalate. Key features within scope include:

- Role-based access control tailored for Android, with secure profiles for Principal, HOD, Faculty and others.
- Academic tools for building courses, semesters, classes and faculty assignments on the go.
- Full examination cycle: scheduling, assessments and result processing.

- Touch-based attendance entry with synced, centralized views.
- Department and class management, including member assignments.

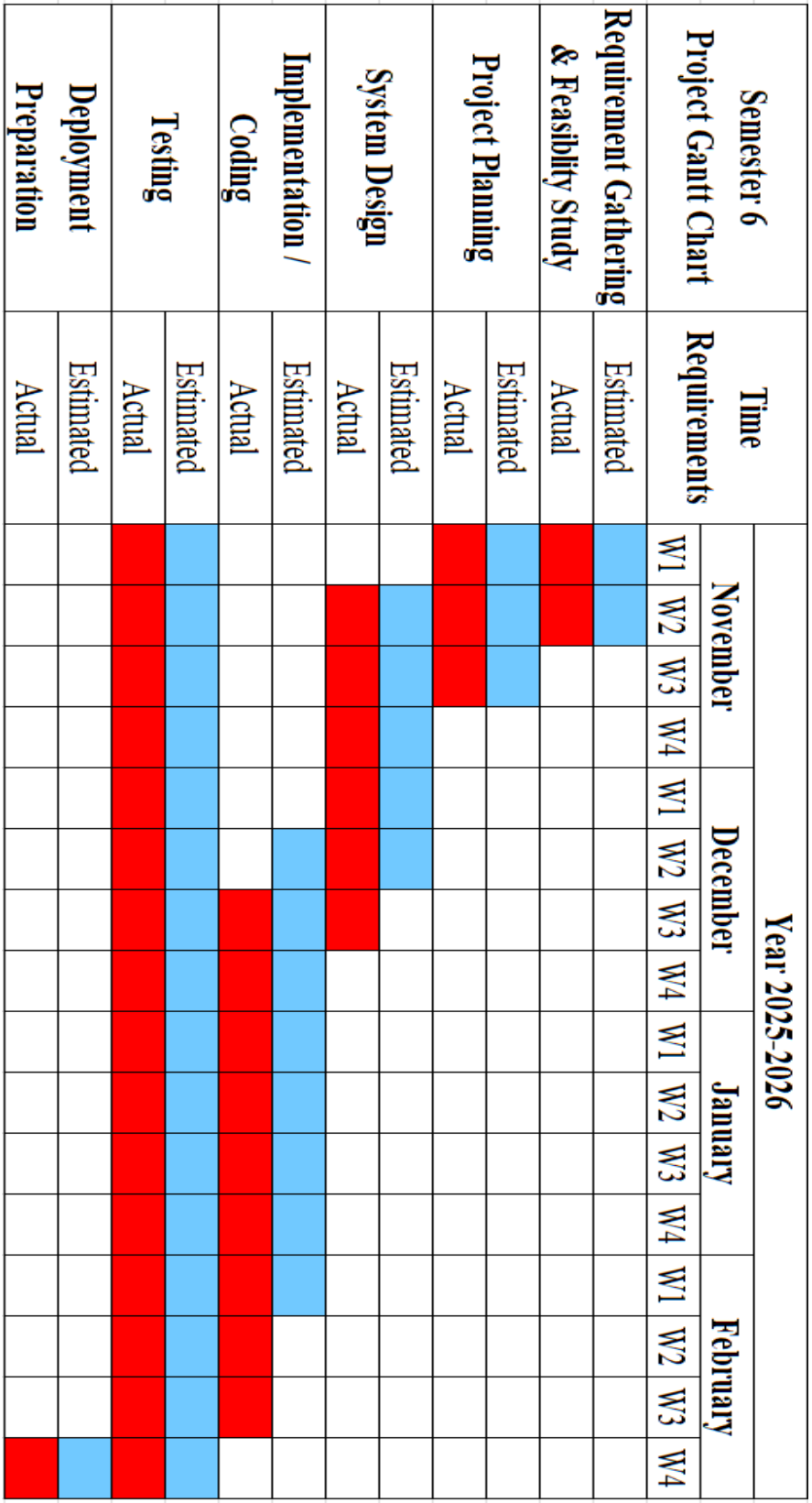
This Android-centric scope targets essential college functions, skipping complex financials or external links for simplicity, while building in modularity for future enhancements like cloud sync.

1.4 Phase Title

Phase Title	Expected Date of Completion	Actual Time of Completion with Guide's Signature	Remarks
I. Introduction			
(i)Background and Project Overview			
(ii) Objectives			
(iii) Purpose & Scope			
(iv) Phase Title			
(v) Gantt Chart			
II. System Analysis			
(i) Existing System			
(ii) Proposed System			
(iii) Requirement Analysis			
III. System Design			
(i) Data Flow Diagram			
(ii) Activity Diagram			
(iii) Sequence Diagram			
(iv) Class Diagram			
(v) Use Case Diagram			
(vi) ER Diagram			
IV. Implementation and Testing			
(i) System Coding			
(ii) Testing Approach			

V. Results			
VI. Conclusion and Future work			
VII. References			
(i) Plagiarism Report			
(ii) Appendix			
(iii) Approved Project Proposal			

1.5 Gantt Chart



CHAPTER 2

SYSTEM ANALYSIS

2.1 Existing Systems

Modern college management systems were introduced to simplify academic and administrative work, but in practice, many still fail to offer a truly smooth and connected experience. Institutions today rely on a mix of solutions such as standalone desktop software, cloud-based ERP systems and basic mobile applications. While each approach brings certain advantages, none fully addresses the everyday operational needs of colleges.

In many institutions, routine tasks such as attendance tracking, exam scheduling and student record maintenance still depend on manual spreadsheets or outdated software. Tools like Excel-based trackers or legacy accounting systems are easy to adopt and work reasonably well for small datasets. However, as student numbers grow, these methods become inefficient. They are highly error-prone, do not update in real time and require repeated manual data entry. During high-pressure periods such as semester-end evaluations or result processing this significantly increases staff workload and the risk of inconsistent or inaccurate records.

Web-based Management Information Systems (MIS) attempt to centralize operations by offering browser-based access to student data, fee management and basic reporting. These platforms function well in structured environments with stable internet connectivity and allow multiple users such as principals and HODs to work simultaneously. Despite this, they often lack flexibility. Limited offline support, poor mobile optimization and insufficient role-based customization make them inconvenient for faculty members or issues or complex permissions hierarchies frequently results in delayed updates, access conflicts or incomplete data synchronizations.

To address mobility, some institutions adopt simple Android applications for specific tasks such as QR-code-based attendance or timetable viewing. These apps are affordable, easy to use and require no additional hardware beyond a smartphone. While effective for individual functions, they operate in isolation. Each app handles only a single task, leading to fragmented data spread across multiple platforms. As a result, staff must manually reconcile information between systems, increasing the chances of duplicate entries, miscommunication between departments and delays in coordinating academic activities, events and examinations.

2.2 Proposed Systems

1. System Architecture

SYNAPSE is designed as a flexible, modular Android application that brings all essential college operations into a single, easy-to-use platform. It is built specifically for administrators, faculty members and staff who need quick access to information while on the move. Developed using Kotlin within the Android ecosystem, the application provides a clean, role-based interface that adapts to different users, whether they are logging in, managing classes or moderating exams directly from their smartphones without need for complex hardware or setups.

Most of the system's core processing is handled directly on the device. Everyday academic tasks such as entering attendance, managing course details, scheduling exams are processed locally using lightweight logic and counter-verified by Firebase databases. This ensures fast response times and allows the app to remain functional even with limited internet connectivity.

To support data security and long-term reliability, SYNAPSE is backed by a Firebase cloud infrastructure. The cloud handles secure authentication, synchronizes data across devices and stores important records such as exam results, attendance reports and new student applications. Sensitive information such as attendance records and exam results, is primarily processed on the device, with only essential data transmitted to the cloud. This approach balances privacy, performance and scalability making SYNAPSE a dependable and future-ready solution for college management

2. Modules

1. LoginActivity.kt

Acts as the authentication gateway for the SYNAPSE MIS platform, offering secure, role-based login with animated orbit visuals..

- **Multi-Role Authentication:** Supports Student, Faculty, HOD, and COE, redirecting each to their dashboard.
- **Input Validation:** Checks email format and password length with inline error feedback.
- **New User Direction:** Guides first-time Faculty through credential setup and enables new Student registration.

2. StudentDashboardActivity.kt

The central hub for student's post-login, offering a personalized academic overview with animated stars and a navigation drawer.

- **Student Profile Display:** Shows name, roll number, semester, class, attendance, and CGPA from Firestore.
- **Quick Action Cards:** Access Attendance, Marks, Timetable, Profile, Assignments, and Class enrollment, with class-aware routing.
- **Class Management:** Detects class membership, prompting code entry if unassigned or showing current class info.
- **Navigation Drawer:** Slide-out menu with feature links, personalized header, and logout.

3. FacultyDashboardActivity.kt

Faculty dashboard providing a real-time summary of workload and access to all faculty functions. Dual-ID logic ensures accurate timetable and attendance mapping.

- **Stats Overview:** Shows assigned subjects, today's lectures, pending tasks, and department from Firestore.
- **Dual-ID Resolution:** Uses Firebase Auth UID and Firestore faculty ID for matching, with email fallback if UID fails.

- **Quick Action Cards:** Access My Subjects, Mark/View Attendance, Timetable, Marks Entry, and Session Round with correct IDs.

4. HODDashboardActivity.kt

HOD dashboard presenting full department details and management tools, with both direct navigation and self-loading if no department ID is provided.

- **Department Profile View:** Displays image, name, code, college, stream, HOD contact, faculty/student count, established year, and description.
- **HOD Action Cards:** Manage Faculty, Students, Attendance Reports, Timetable, Leave Approvals, Classes, and Department Stats.
- **Self-Loading Logic:** Auto-fetches the department from Firestore if no DEPARTMENT_ID is passed.

5. CoeDashboardActivity.kt

COE dashboard providing an institution-wide overview and access to exam, attendance, and timetable management.

- **Institution-Wide Stats:** Shows totals for students, faculty, departments, classes, average attendance, exam marks, and pending complaints from Firestore.
- **Quick Action Cards:** Access Attendance Reports, Exam Management, and Timetable List.
- **Navigation Drawer:** Links to Departments, Students, Faculty, Attendance, Exams, Timetable, Application Review, with a COE-profile header..

6. ExamManagementActivity.kt

COE exam hub offering card-based access to all major exam workflows in SynapseMIS..

- **Create Exam:** Launches multi-step exam setup.
- **Manage Exams:** View, edit, and control existing exams..
- **Exam History:** Browse a filterable archive of past exams.
- **Result Generation and Mark Tracking:** Generate student results and monitor marks entry across subjects.

7. ExamCreationStep1Activity.kt, ExamCreationStep2Activity.kt, ExamCreationStep3Activity.kt, ExamCreationStep4Activity.kt

Four-step COE exam setup wizard covering course configuration, subject scheduling, and final review.).

- **Course Selection and Configuration (Step 2):** Fetches courses from Firestore; set student type, exam type, and max marks; edit or remove before proceeding.
- **Subject Scheduling (Step 3):** Assign per-subject date, start time, and duration with date/time pickers.
- **Review and Confirmation (Step 4):** Displays exam summary and timetable preview; confirms submission to Firestore.
- **Post-Creation Navigation:** Navigate to exam details or return to exams list after successful creation.

8. FacultyMarksEntryActivity.kt

Faculty marks entry interface for inputting written and internal exam marks, with draft saving and formal submission tracking.

- **Student Marks List:** Loads enrolled students via RecyclerView with fields per exam type.
- **Split Marks Schema:** Supports written-only, internal-only, or combined marks with auto-calculated grades.
- **Save Draft and Submit:** Drafts can be saved; submission locks entry and updates Firestore for HOD review.
- **Warning and Status Display:** Alerts for closing windows or already submitted marks; shows completion progress.

9. HodMarksModerationActivity.kt

T HOD marks moderation interface for reviewing, filtering, and adjusting faculty-submitted marks before COE publication.

- **Marks Review:** Displays subject exam marks with per-student editing via RecyclerView.

- **Advanced Filtering:** Filter by marks range, roll number, name, absent or modified status, with sorting options.
- **Moderation Submission:** Save drafts or submit to mark the subject as moderated for COE publishing.

10.CoeMarksPublishActivity.kt

COE marks publication interface for reviewing subject moderation status and publishing results to students.

- **Publication Status Overview:** Shows total, moderated, and pending subjects for a clear readiness snapshot.
- **Per-Subject Status List:** Displays each subject's status (Entered, Submitted, Moderated, Published) via RecyclerView.
- **Selective Publishing:** Publish individual subjects or batch publish all moderated subjects using Firestore WriteBatch.
- **COE Access Verification:** Confirms COE role in Firestore before granting access.

11. GenerateResultActivity.kt

COE result compilation interface for generating consolidated student results from published exams.

- **Filter Selection:** Department, course, and semester spinners narrow scope before loading exams.
- **Exam Type Grouping:** Select Written, Internal, and optional Practical exams via chip groups for inclusion.
- **Result Generation:** Combines selected exam marks per student to produce final results.
- **Selection Status Feedback:** Dynamic status view guides minimum selections before generation.

12.ResultDisplayActivity.kt

Result gazette viewer consolidating multiple exam types per student and enabling PDF export.

- **Multi-Exam Aggregation:** Result gazette viewer consolidating multiple exam types per student and enabling PDF export.
- **Result Gazette Adapter:** RecyclerView via ResultGazetteAdapter shows per-student subject breakdowns, total marks, CGPI, and result class.
- **Stats Bar:** Summarizes total students, passed/failed count, and average CGPI.
- **PDF Export:** Generates A4 landscape PDF with share option via ResultPdfGenerator.

13.CreateTimetableActivity.kt

Two-step timetable builder for administrators to select classes and schedule day-by-day lecture and break slots..

- **Step 1 Class Selection:** Cascading Department, Course, and Class spinners load Firestore data; default lecture duration set.
- **Step 2 Day Scheduling:** Day selector switches RecyclerView for Monday–Saturday slots, each with independent Timetable Slot lists.
- **Add Lecture and Break:** Dialogs allow adding lectures (subject, faculty, time) or breaks (label, duration).
- **Save Timetable:** Saves full schedule to Firestore and returns to TimetableListActivity.

14.StudentAttendanceAnalyticsActivity.kt

Advanced attendance analytics for HOD and COE, offering class-level insights with charts, risk classification, and AI-generated summaries.

- **Date Range Filter:** Start/end pickers (default current month) with Apply button refresh calculations.
- **Custom Chart Visualizations:** Bar chart (date-wise), pie chart (present/absent), and sparkline (monthly per-student trends).
- **Risk Classification:** Students labeled ‘At Risk’, ‘Warning’, or ‘Good’ with summary counts.

15.DefaulterListActivity_COE.kt

COE defaulter report generator identifying students below a configurable attendance threshold for selected classes and date ranges.

- **Class and Period Selection:** Class spinner and start/end pickers (default current month) define analysis scope.
- **Threshold-Based Identification:** Flags students below the configurable cutoff (default 75%) with per-subject calculations.
- **Defaulter List with Shortfalls:** RecyclerView shows roll number, name, and subject shortfalls sorted by attendance.
- **Detail Navigation:** Tap a student to view full attendance history in StudentAttendanceAnalysisActivity.

16.TrackApplicationActivity.kt

Public application tracker for prospective students to check admission status via a unique reference number.

- **Reference Number Lookup:** Queries Firestore by reference number; supports auto-fill via intent.
- **Status Display:** Shows applicant name, email, course, application date, and dynamic status (Pending, Accepted, Rejected, Enrolled) with icons and color codes.
- **Status Messages:** Contextual messages guide next steps for accepted accounts or display rejection reasons.

3. Data Handling

SYNAPSE MIS leverages Cloud Firestore for comprehensive data management, providing real-time synchronization, secure authentication and reliable cloud storage. The system implements multiple layers of data security including role-based access control for sensitive academic records, Firebase Authentication tied to verified user governing what each role (student, faculty or HOD) can read or modify. Local data caching ensures full functionality during poor connectivity while maintaining seamless synchronization when the connection is restored.

Student profiles, attendance records, examination marks and enrollment data are stored with appropriate privacy controls, allowing institutions to maintain ownership of their data while enabling essential academic operations. Sensitive information such as parent contact details, date of birth and academic performance history is scoped strictly to authorized roles, ensuring that personal data is never exposed beyond its intended audience.

4. Deployment and Technology Stack

Synapse is developed using **Android Studio**, leveraging **Kotlin** to deliver smooth native Android performance along with full access to device-level features. This approach ensures that the application remains fast, responsive and compatible with a wide range of Android devices commonly used by faculty members and administrative staff. Native development allows Synapse to efficiently utilize smartphone capabilities without relying on external hardware or complex installations.

For backend support, Synapse integrates **Firebase** to manage secure user authentication, cloud-based data storage and real-time data synchronization. Firebase enables seamless login handling, role-based access verification and reliable syncing of academic and administrative records across devices. Data entered during offline usage such as attendance or event updates is automatically synchronized once an internet connection becomes available, ensuring consistency without manual intervention.

On the frontend, Synapse follows **Material Design principles**, providing a clean, intuitive and consistent user interface across all modules. Role-based layouts ensure that users only see features relevant to their responsibilities, reducing clutter and improving usability. Background

services are carefully managed to support data synchronization without affecting device performance.

Overall, the chosen technology stack offers a balanced combination of performance, security, scalability and user experience. It allows Synapse to function reliably in both online and near-offline environments while remaining flexible enough to support future enhancements such as analytics, reporting dashboards or insights.

5. Evaluation

Synapse underwent testing across simulated college scenarios to verify core reliability under typical institutional conditions, such as varying network availability and user roles. Key safety and access features like role-based logins and data entry were validated through basic controlled checks, confirming high accuracy in tasks like attendance marking and exam scheduling.

User feedback from initial trails with mock faculty and admin profiles guided minor UI tweaks and algorithm reojfinements for smoother navigation. Performance assessments ensured the Android app runs efficiently on mid-range devices, maintain responsive monitoring of academic workflows without lags or delays.

Ongoing evaluations incorporate this feedback for iterative updates, focusing on algorithm stability and emerging needs, keeping SYNAPSE practical and effective for real-world college deployment.

2.2 Requirement Analysis

2.2.1 Hardware Requirements & it's specifications:

Component	**Recommended Specifications**
Android Smartphone	Android 8 or above, Minimum 3 GB RAM
Internet Connectivity	Stable broadband (>= 5 Mbps)

2.2.2 Software Requirements & it's specifications

Software Components:	**Required Version:**
Android Studio	Latest stable version (>= Flamingo)
Firebase	Blaze plan (for scalable usage)
Programming Languages	Kotlin 1.9 or above, Python 3.10 or above

CHAPTER 3

SYSTEM DESIGN

3.1 Use Case Diagram

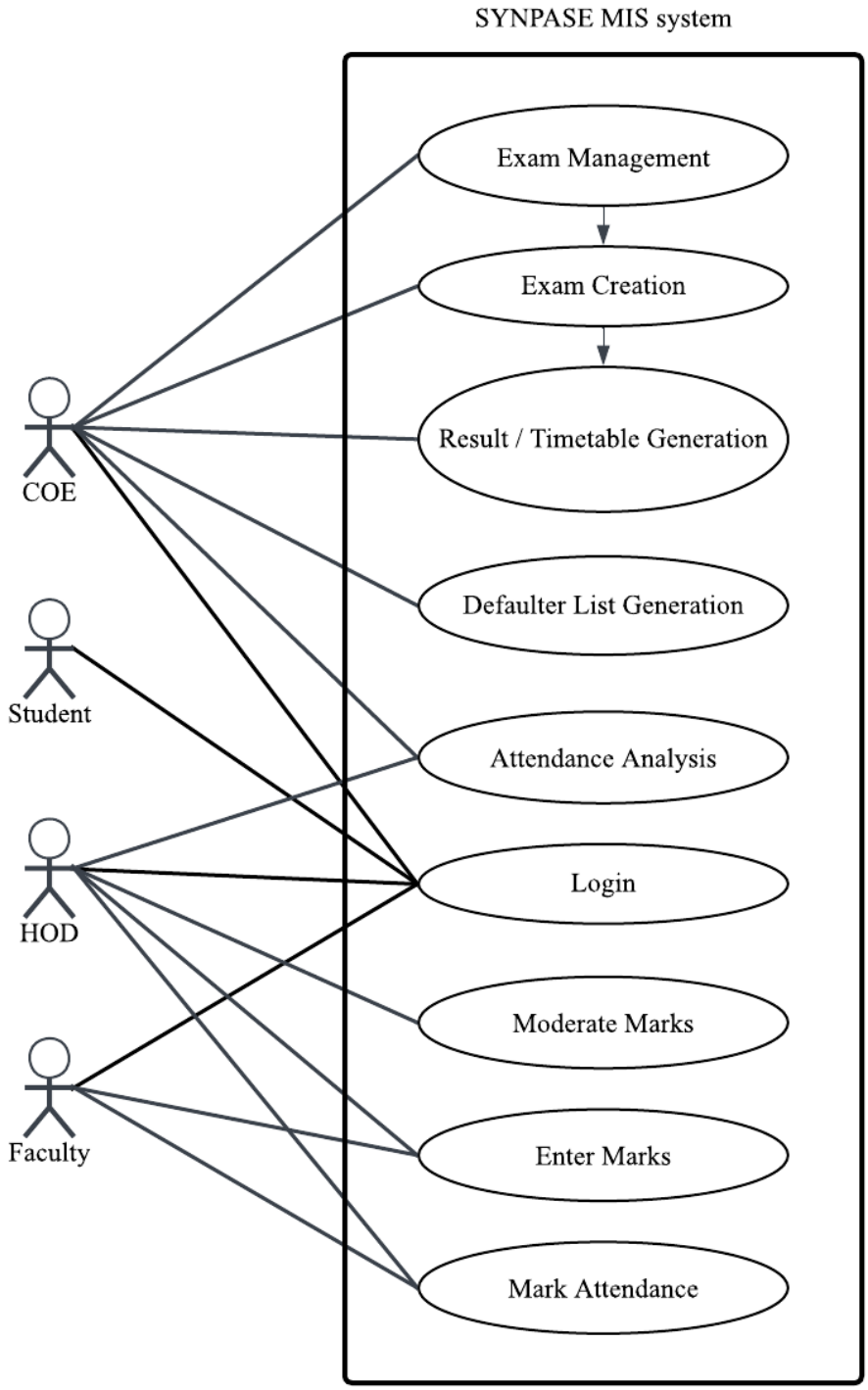


Fig 3.1 Use Case Diagram

3.2 Activity Diagram

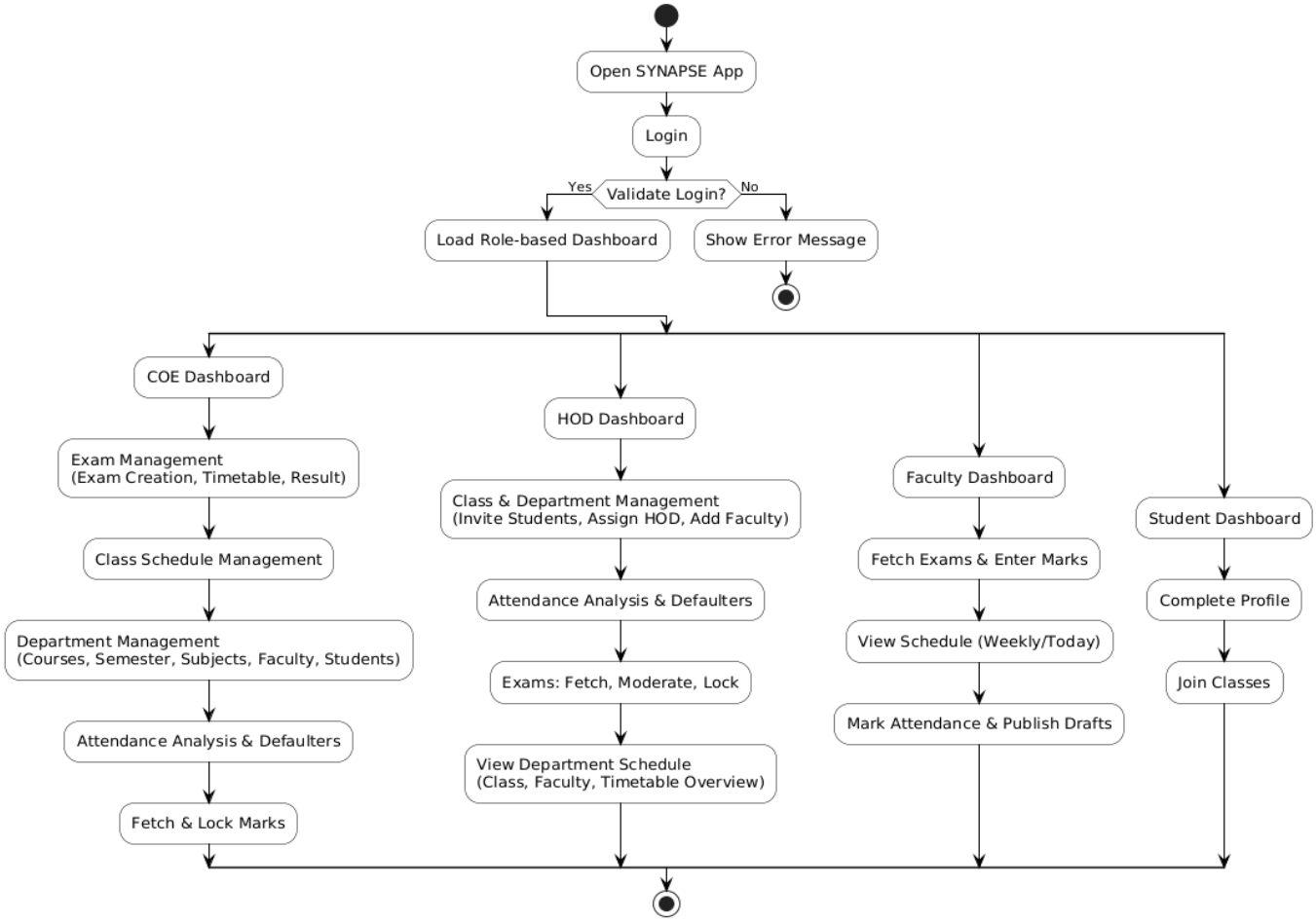


Fig 3.2 Activity Diagram

3.3 Sequence Diagram

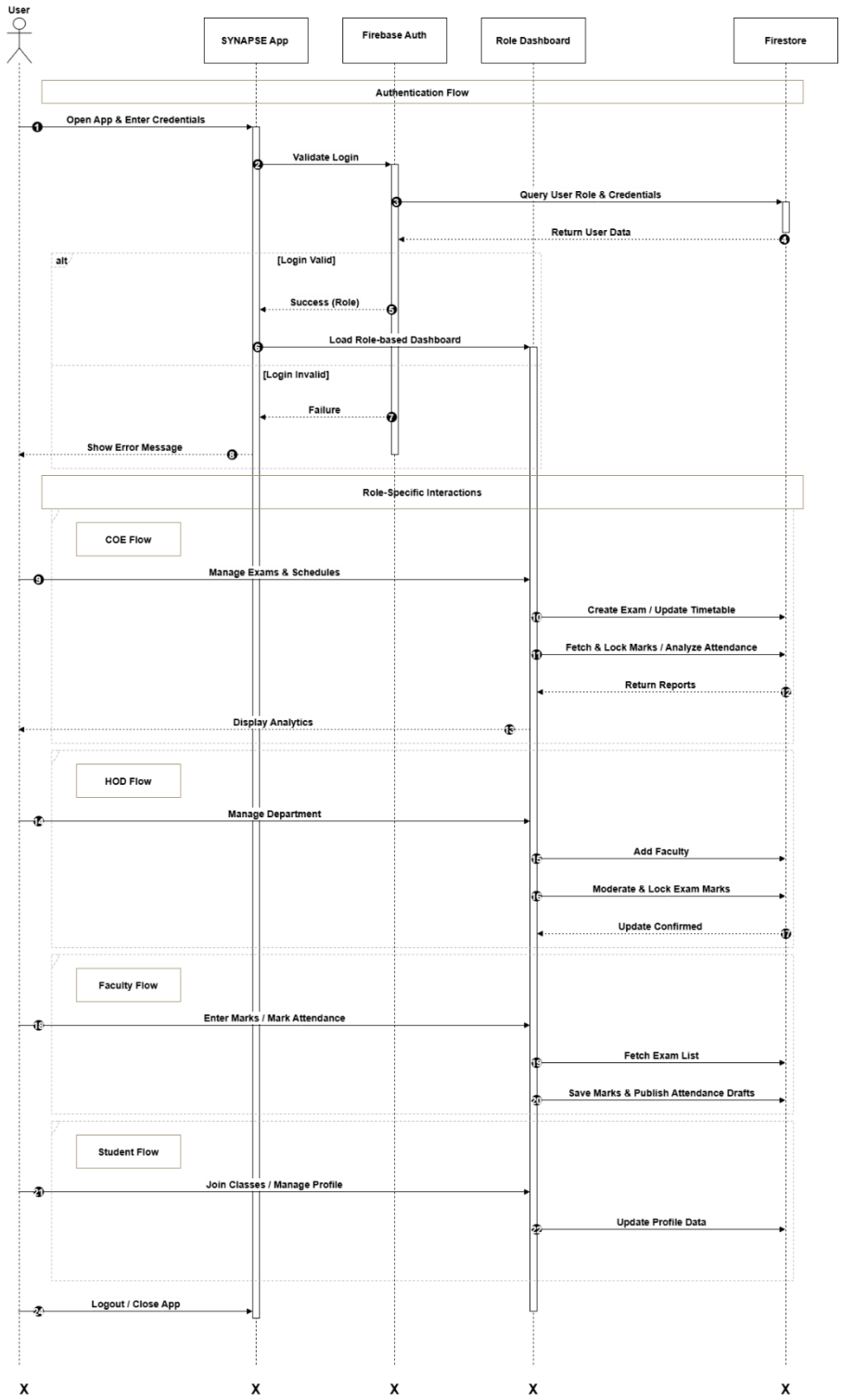


Fig 3.3 Sequence Diagram

3.4 Class Diagram

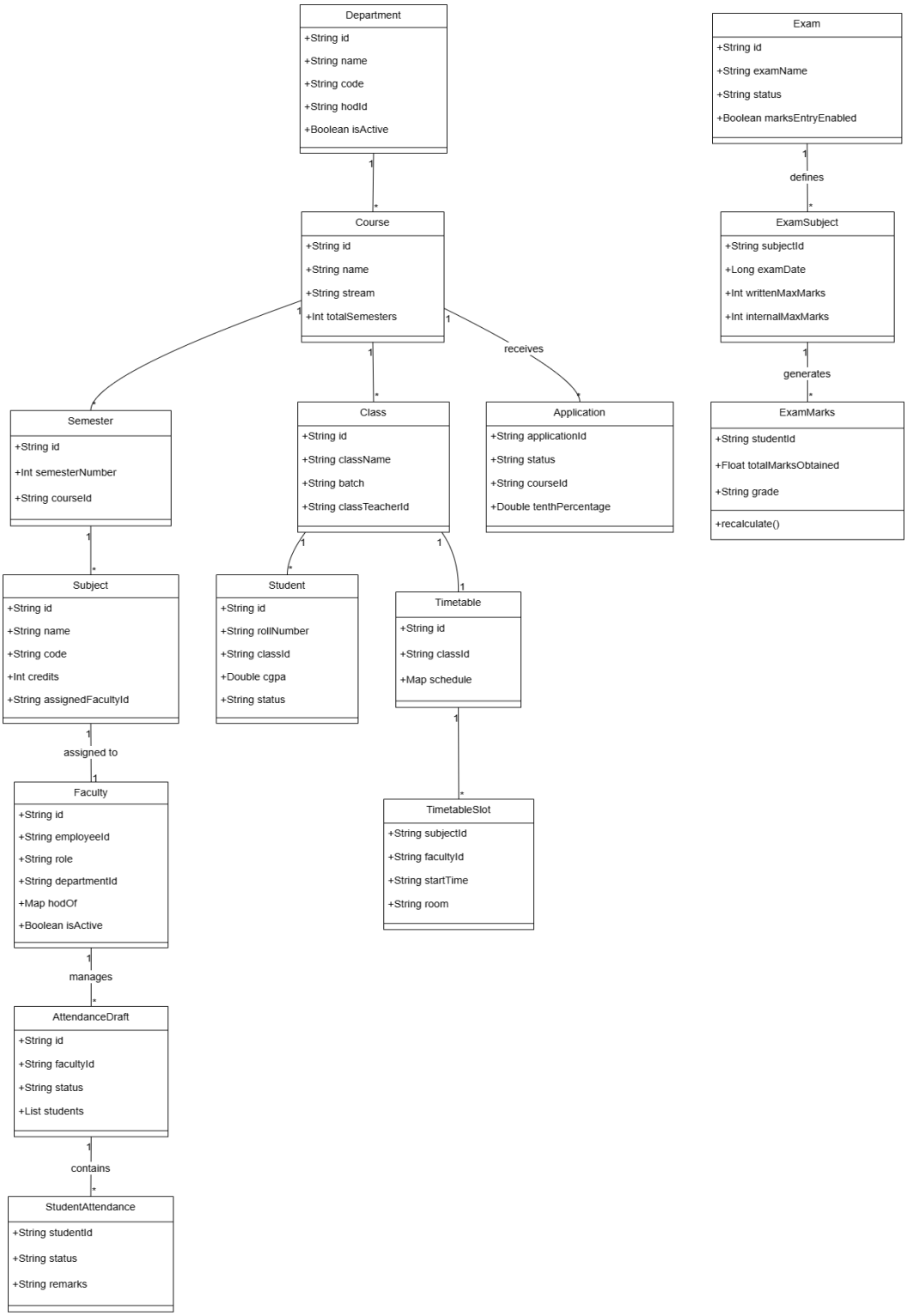


Fig 3.4 Class diagram

3.5 ER Diagram

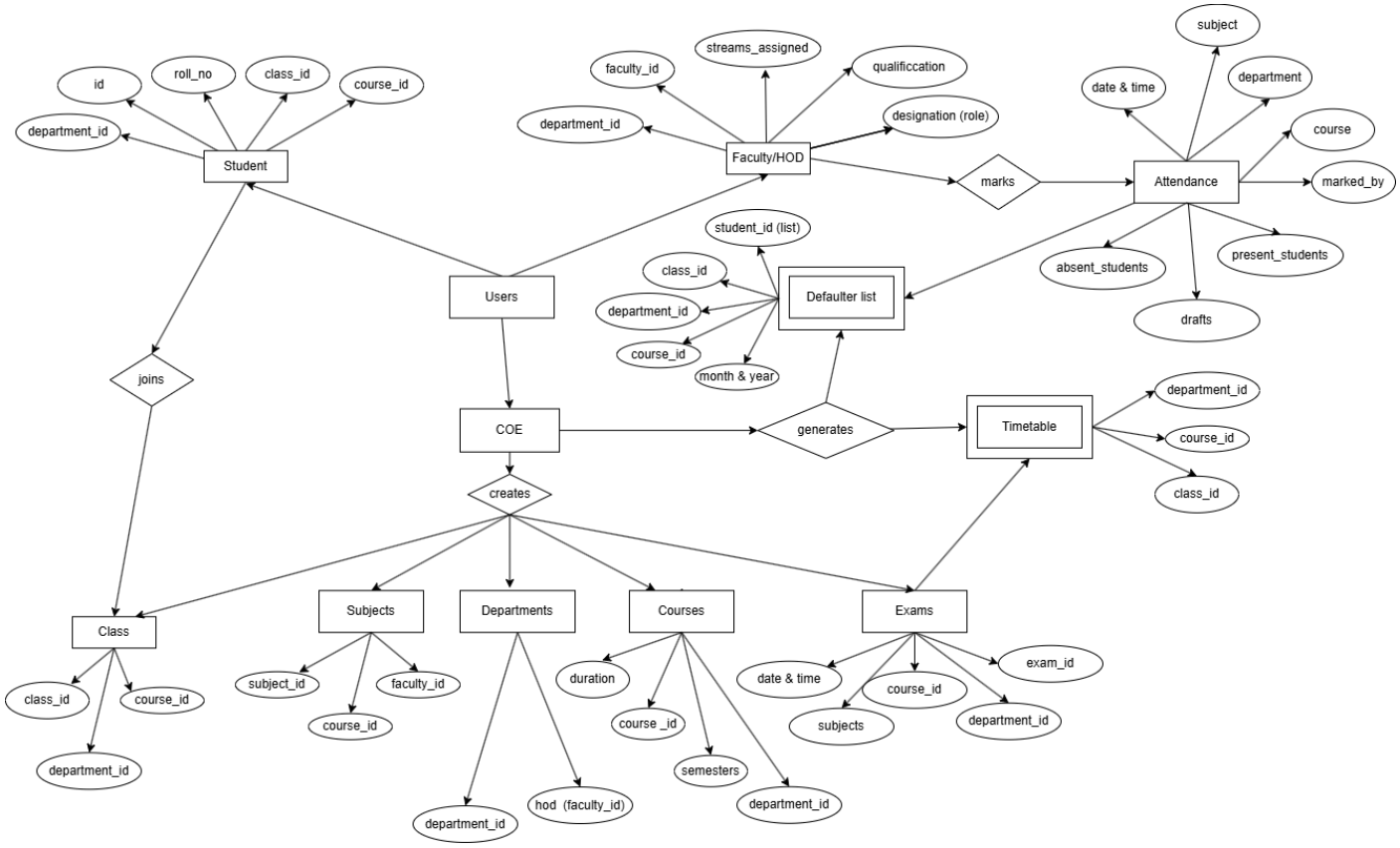


Fig 3.5 ER diagram

3.6 Data Flow Diagram

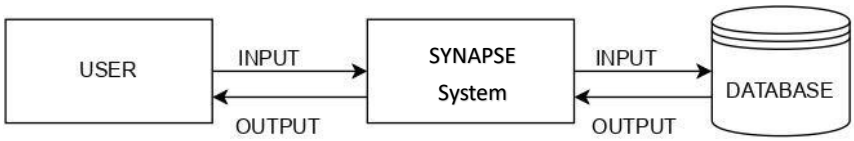


Fig 3.6.1 Data flow diagram level 0

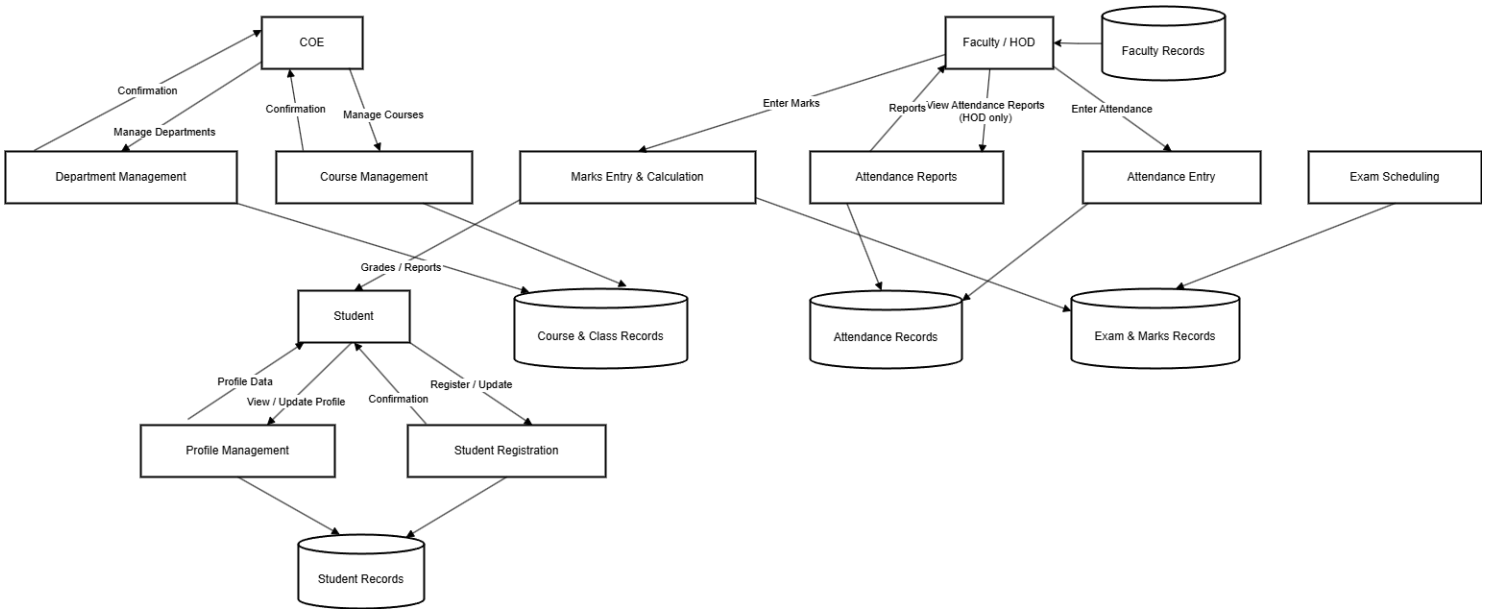


Fig 3.6.2 Data flow diagram level 1

CHAPTER 4

IMPLEMENTATION AND TESTING

4.1 Code

- **LoginActivity.kt**

```
package com.projectbyyatin.synapsemis

import android.animation.ObjectAnimator
import android.content.Intent
import android.os.Bundle
import android.util.Patterns
import android.view.View
import android.view.animation.LinearInterpolator
import android.widget.ProgressBar
import android.widget.TextView
import android.widget.Toast
import androidx.appcompat.app.AppCompatActivity
import com.google.android.material.button.MaterialButton
import com.google.android.material.textfield.TextInputEditText
import com.google.android.material.textfield.TextInputLayout
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.firestore.FirebaseFirestore

class LoginActivity : AppCompatActivity() {
private lateinit var auth: FirebaseAuth
private lateinit var firestore: FirebaseFirestore
private lateinit var emailInput: TextInputEditText
private lateinit var passwordInput: TextInputEditText
private lateinit var emailInputLayout: TextInputLayout
private lateinit var passwordInputLayout: TextInputLayout
private lateinit var loginButton: MaterialButton
private lateinit var forgotPasswordText: TextView
private lateinit var firstLoginText: TextView
```

```
private lateinit var loginProgress: ProgressBar
private lateinit var orbit1: View
private lateinit var orbit2: View
private lateinit var orbit3: View

companion object {
const val ROLE_STUDENT = "student"
const val ROLE_FACULTY = "faculty"
const val ROLE_HOD = "hod"
const val ROLE_COE = "coe"
}

override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_login)
supportActionBar?.hide()
auth = FirebaseAuth.getInstance()
firestore = FirebaseFirestore.getInstance()
initializeViews()
startOrbitAnimations()
setupClickListeners()
}
private fun initializeViews() {
emailInput = findViewById(R.id.email_input)
passwordInput = findViewById(R.id.password_input)
emailInputLayout = findViewById(R.id.email_input_layout)
passwordInputLayout = findViewById(R.id.password_input_layout)
loginButton = findViewById(R.id.login_button)
forgotPasswordText = findViewById(R.id.forgot_password_text)
firstLoginText = findViewById(R.id.first_login_text)
loginProgress = findViewById(R.id.login_progress)
```

```
orbit1 = findViewById(R.id.orbit1)
orbit2 = findViewById(R.id.orbit2)
orbit3 = findViewById(R.id.orbit3) }
private fun startOrbitAnimations() {
listOf(
Pair(orbit1, 20000L),
Pair(orbit2, 15000L),
Pair(orbit3, 10000L)
).forEachIndexed { index, pair ->
ObjectAnimator.ofFloat(
pair.first,
"rotation",
if (index == 1) 360f else 0f,
if (index == 1) 0f else 360f).apply {
duration = pair.second
repeatCount = ObjectAnimator.INFINITE
interpolator = LinearInterpolator()
start()}}}
private fun setupClickListeners() {
loginButton.setOnClickListener { attemptLogin() }
forgotPasswordText.setOnClickListener { showForgotPasswordDialog() }
firstLoginText.setOnClickListener {
startActivity(Intent(this, FacultyFirstLoginActivity::class.java))}}
private fun attemptLogin() {
emailInputLayout.error = null
passwordInputLayout.error = null
val email = emailInput.text.toString().trim()
val password = passwordInput.text.toString().trim()
if (!validateInputs(email, password)) return
showLoading(true)
auth.signInWithEmailAndPassword(email, password)
```

```
.addOnSuccessListener { navigateBasedOnRole() }
.addOnFailureListener {
showLoading(false)
Toast.makeText(this, it.message, Toast.LENGTH_LONG).show()}}
private fun validateInputs(email: String, password: String): Boolean {
var valid = true
if (email.isEmpty() || !Patterns.EMAIL_ADDRESS.matcher(email).matches()) {
emailInputLayout.error = "Valid email required"
valid = false}
if (password.length < 6) {
passwordInputLayout.error = "Minimum 6 characters"
valid = false}
return valid}
private fun navigateBasedOnRole() {
val user = auth.currentUser ?: return
firestore.collection("users").document(user.uid).get()
.addOnSuccessListener { userDoc ->
if (!userDoc.exists()) {
blockLogin("User profile not found")
return@addOnSuccessListener}
val role = userDoc.getString("role")?.lowercase()
val completed = userDoc.getBoolean("profileCompleted") ?: false
when (role) {
ROLE_STUDENT -> handleStudent(user.uid, completed)
ROLE_FACULTY, ROLE_HOD -> handleFaculty(userDoc, completed)
ROLE_COE -> navigateToDashboard(ROLE_COE)
else -> blockLogin("Invalid role")}}
.addOnFailureListener { blockLogin(it.message ?: "Error") }}
private fun handleStudent(userId: String, completed: Boolean) {
if (completed) navigateToDashboard(ROLE_STUDENT)
else navigateToProfileSetup(userId)
```

```
private fun handleFaculty(userDoc: com.google.firebase.firestore.DocumentSnapshot,
completed: Boolean) {
val facultyId = userDoc.getString("facultyId") ?: run {
blockLogin("Faculty mapping missing"); return}
firestore.collection("faculty").document(facultyId).get()
.addOnSuccessListener { facultyDoc ->
if (!facultyDoc.exists()) {
blockLogin("Faculty record not found")
return@addOnSuccessListener}
if (!completed) {
navigateToFacultyProfileSetup(facultyId)
return@addOnSuccessListener}
when (facultyDoc.getString("role")?.lowercase()) {
ROLE_HOD -> navigateToDashboard(ROLE_HOD)
ROLE_FACULTY -> navigateToDashboard(ROLE_FACULTY)
else -> blockLogin("Invalid faculty role")}}
.addOnFailureListener { blockLogin(it.message ?: "Faculty error") }}
private fun navigateToProfileSetup(userId: String) {
showLoading(false)
startActivity(Intent(this, StudentProfileSetupActivity::class.java).apply {
putExtra("USER_ID", userId)
flags = Intent.FLAG_ACTIVITY_NEW_TASK or
Intent.FLAG_ACTIVITY_CLEAR_TASK})
finish()}
private fun navigateToFacultyProfileSetup(facultyId: String) {
showLoading(false)
startActivity(Intent(this, FacultyProfileSetupActivity::class.java).apply {
putExtra("FACULTY_ID", facultyId)
flags = Intent.FLAG_ACTIVITY_NEW_TASK or
Intent.FLAG_ACTIVITY_CLEAR_TASK})
finish()}
```

```
private fun navigateToDashboard(role: String) {
showLoading(false)
val intent = when (role) {
ROLE_COE -> Intent(this, CoeDashboardActivity::class.java)
ROLE_HOD -> Intent(this, HodDashboardActivity::class.java)
ROLE_FACULTY -> Intent(this, FacultyDashboardActivity::class.java)
ROLE_STUDENT -> Intent(this, StudentDashboardActivity::class.java)
else -> return}
intent.flags = Intent.FLAG_ACTIVITY_NEW_TASK or
Intent.FLAG_ACTIVITY_CLEAR_TASK
startActivity(intent)
finish()}
private fun blockLogin(message: String) {
showLoading(false)
Toast.makeText(this, message, Toast.LENGTH_LONG).show()
auth.signOut()}
private fun showForgotPasswordDialog() {
Toast.makeText(this, "Password reset handled elsewhere", Toast.LENGTH_SHORT).show()}
private fun showLoading(show: Boolean) {
loginProgress.visibility = if (show) View.VISIBLE else View.GONE
loginButton.isEnabled = !show
}
override fun onStart() {
super.onStart()
if (auth.currentUser != null) navigateBasedOnRole()}}
```

- **CoeDashboardActivity.kt**

```
package com.projectbyyatin.synapsemis

import android.animation.ObjectAnimator
import android.animation.ValueAnimator
```

```
import android.content.Intent
import android.os.Bundle
import android.view.MenuItem
import android.widget.*
import androidx.activity.OnBackPressedCallback
import androidx.appcompat.app.ActionBarDrawerToggle
import androidx.appcompat.app.AppCompatActivity
import androidx.appcompat.widget.Toolbar
import androidx.core.view.GravityCompat
import androidx.drawerlayout.widget.DrawerLayout
import com.google.android.material.card.MaterialCardView
import com.google.android.material.navigation.NavigationView
import com.google.firebase.auth.FirebaseAuth
import com.google.firebase.firestore.FirebaseFirestore

class CoeDashboardActivity : AppCompatActivity(),
NavigationView.OnNavigationItemSelectedListener {
private lateinit var drawerLayout: DrawerLayout
private lateinit var navigationView: NavigationView
private lateinit var toolbar: Toolbar
private lateinit var auth: FirebaseAuth
private lateinit var firestore: FirebaseFirestore
private lateinit var totalStudentsText: TextView
private lateinit var totalFacultyText: TextView
private lateinit var totalDepartmentsText: TextView
private lateinit var totalClassesText: TextView
private lateinit var avgAttendanceText: TextView
private lateinit var avgExamMarksText: TextView
private lateinit var complaintsCountText: TextView
private lateinit var btnAttendance: MaterialCardView
private lateinit var btnExams: MaterialCardView
```

```
private lateinit var btnTimetable: MaterialCardView

override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_coe_dashboard)
auth = FirebaseAuth.getInstance()
firestore = FirebaseFirestore.getInstance()
initializeViews()
setupToolbar()
setupNavigationDrawer()
loadDashboardData()
animateStars()
setupClickListeners()
onBackPressedDispatcher.addCallback(this, object : OnBackPressedCallback(true) {
override fun handleOnBackPressed() {
if (drawerLayout.isDrawerOpen(GravityCompat.START)) {
drawerLayout.closeDrawer(GravityCompat.START)
} else {
isEnabled = false
onBackPressedDispatcher.onBackPressed()
isEnabled = true}}})}
private fun initializeViews() {
drawerLayout = findViewById(R.id.drawer_layout)
navigationView = findViewById(R.id.nav_view)
toolbar = findViewById(R.id.toolbar)
totalStudentsText = findViewById(R.id.total_students)
totalFacultyText = findViewById(R.id.total_faculty)
totalDepartmentsText = findViewById(R.id.total_departments)
totalClassesText = findViewById(R.id.total_classes)
avgAttendanceText = findViewById(R.id.avg_attendance)
avgExamMarksText = findViewById(R.id.avg_exam_marks)
```

```
complaintsCountText = findViewById(R.id.complaints_count)
btnAttendance = findViewById(R.id.btn_attendance)
btnExams = findViewById(R.id.btn_exams)
btnTimetable = findViewById(R.id.btn_timetable) }
private fun setupToolbar() {
setSupportActionBar(toolbar)
supportActionBar?.setDisplayHomeAsUpEnabled(true) }
private fun setupNavigationDrawer() {
val toggle = ActionBarDrawerToggle(
this,
drawerLayout,
toolbar,
R.string.navigation_drawer_open,
R.string.navigation_drawer_close)
drawerLayout.addDrawerListener(toggle)
toggle.syncState()
navigationView.setNavigationItemSelectedListener(this)
updateNavigationHeader()
private fun updateNavigationHeader() {
val header = navigationView.getHeaderView(0)
val name = header.findViewById<TextView>(R.id.nav_header_name)
val email = header.findViewById<TextView>(R.id.nav_header_email)
val role = header.findViewById<TextView>(R.id.nav_header_role)
auth.currentUser?.let { user ->
firestore.collection("users").document(user.uid).get()
.addOnSuccessListener {
if (it.exists()) {
name.text = it.getString("name") ?: "Vice Principal"
email.text = it.getString("email") ?: "coe@college.com"
role.text = "Controller of Examinations"}}}}
private fun loadDashboardData() {
```

```
firestore.collection("students").get()
.addOnSuccessListener { totalStudentsText.text = it.size().toString()
firestore.collection("faculty").get()
.addOnSuccessListener { totalFacultyText.text = (it.size() / 2).toString() }
firestore.collection("departments").get()
.addOnSuccessListener { totalDepartmentsText.text = it.size().toString()
firestore.collection("classes").get()
.addOnSuccessListener { totalClassesText.text = it.size().toString() }
firestore.collection("attendance").get()
.addOnSuccessListener { docs ->
if (!docs.isEmpty) {
val avg = docs.sumOf { it.getDouble("percentage") ?: 0.0 } / docs.size()
avgAttendanceText.text = String.format("%.1f%%", avg)}
firestore.collection("exams").get()
.addOnSuccessListener { docs ->
if (!docs.isEmpty) {
val avg = docs.sumOf { it.getDouble("marks") ?: 0.0 } / docs.size()
avgExamMarksText.text = String.format("%.1f%%", avg) }}
firestore.collection("complaints")
.whereEqualTo("status", "pending")
.get()
.addOnSuccessListener { complaintsCountText.text = it.size().toString() }}
private fun setupClickListeners() {
btnAttendance.setOnClickListener {
startActivity(Intent(this, AttendanceReportActivity_COE::class.java)) }
btnExams.setOnClickListener {
startActivity(Intent(this, ExamManagementActivity::class.java)) }
btnTimetable.setOnClickListener {
startActivity(Intent(this, TimetableListActivity::class.java)) }}
override fun onNavigationItemSelected(item: MenuItem): Boolean {
when (item.itemId) {
```

```
R.id.nav_departments ->
startActivity(Intent(this, DepartmentsActivity::class.java))
R.id.nav_students ->
startActivity(Intent(this, StudentsActivity::class.java))
R.id.nav_attendance ->
startActivity(Intent(this, AttendanceReportActivity_COE::class.java))
R.id.nav_mark_attendance ->
startActivity(Intent(this, MarkAttendanceCOEActivity::class.java))
R.id.nav_exams ->
startActivity(Intent(this, ExamManagementActivity::class.java))
R.id.nav_timetable ->
startActivity(Intent(this, TimetableListActivity::class.java))
R.id.nav_faculties ->
startActivity(Intent(this, FacultyActivity::class.java))
R.id.nav_applications ->
startActivity(Intent(this, CoeApplicationReviewActivity::class.java))
R.id.nav_logout -> logout()}
drawerLayout.closeDrawer(GravityCompat.START)
return true}
private fun logout() {
androidx.appcompat.app.AlertDialog.Builder(this)
.setTitle("Logout")
.setMessage("Are you sure you want to logout?")
.setPositiveButton("Yes") { _, _ ->
auth.signOut()
startActivity(
Intent(this, MainActivity::class.java).apply {
flags = Intent.FLAG_ACTIVITY_NEW_TASK or
Intent.FLAG_ACTIVITY_CLEAR_TASK)
finish()}
.setNegativeButton("No", null)
```

```
.show()}
private fun animateStars() {
val stars = listOf(
R.id.dashboard_star1, R.id.dashboard_star2, R.id.dashboard_star3,
R.id.dashboard_star4, R.id.dashboard_star5, R.id.dashboard_star6,
R.id.dashboard_star7, R.id.dashboard_star8, R.id.dashboard_star9,
R.id.dashboard_star10, R.id.dashboard_star11, R.id.dashboard_star12
).map { findViewById<ImageView>(it) }
stars.forEachIndexed { i, star ->
ObjectAnimator.ofFloat(star, "alpha", 0.3f, 1f, 0.3f).apply {
duration = 2000 + i * 200L
repeatCount = ValueAnimator.INFINITE
startDelay = i * 300L
start()}}}}
```

- **GenerateResultActivity**

```
package com.projectbyyatin.synapsemis

import android.content.Intent
import android.os.Bundle
import android.view.View
import android.widget.*
import androidx.appcompat.app.AppCompatActivity
import androidx.appcompat.widget.Toolbar
import com.google.android.material.button.MaterialButton
import com.google.android.material.card.MaterialCardView
import com.google.android.material.chip.Chip
import com.google.android.material.chip.ChipGroup
import com.google.firebase.Timestamp
import com.google.firebase.firestore.FirebaseFirestore
import com.projectbyyatin.synapsemis.models.Exam
```

```
import com.projectbyyatin.synapsemis.models.ExamSubject
import com.projectbyyatin.synapsemis.models.SubjectMarksStatus

class GenerateResultActivity : AppCompatActivity() {
private lateinit var toolbar: Toolbar
private lateinit var departmentSpinner: Spinner
private lateinit var courseSpinner: Spinner
private lateinit var semesterSpinner: Spinner
private lateinit var loadExamsButton: MaterialButton
private lateinit var writtenExamsCard: MaterialCardView
private lateinit var writtenExamsChipGroup: ChipGroup
private lateinit var writtenEmptyText: TextView
private lateinit var internalExamsCard: MaterialCardView
private lateinit var internalExamsChipGroup: ChipGroup
private lateinit var internalEmptyText: TextView
private lateinit var practicalExamsCard: MaterialCardView
private lateinit var practicalExamsChipGroup: ChipGroup
private lateinit var practicalEmptyText: TextView
private lateinit var tvPracticalOptional: TextView
private lateinit var generateButton: MaterialButton
private lateinit var loadingProgress: ProgressBar
private lateinit var selectionStatusText: TextView
private val firestore = FirebaseFirestore.getInstance()
private val departments = mutableListOf<DeptItem>()
private val courses = mutableListOf<CourseItem>()
private val publishedExams = mutableListOf<ExamItem>()
private var selectedDeptId = ""
private var selectedCourseId = ""
private var selectedSemester =
private val selectedWrittenExamIds = mutableSetOf<String>()
private val selectedInternalExamIds = mutableSetOf<String>()
```

```
private val selectedPracticalExamIds = mutableSetOf<String>()
override fun onCreate(savedInstanceState: Bundle?) {
super.onCreate(savedInstanceState)
setContentView(R.layout.activity_generate_result)
initViews()
setupToolbar()
loadDepartments()}
private fun initViews() {
toolbar = findViewById(R.id.toolbar)
departmentSpinner = findViewById(R.id.department_spinner)
courseSpinner = findViewById(R.id.course_spinner)
semesterSpinner = findViewById(R.id.semester_spinner)
loadExamsButton = findViewById(R.id.load_exams_button)
writtenExamsCard = findViewById(R.id.written_exams_card)
writtenExamsChipGroup = findViewById(R.id.written_exams_chip_group)
writtenEmptyText = findViewById(R.id.written_empty_text)
internalExamsCard = findViewById(R.id.internal_exams_card)
internalExamsChipGroup = findViewById(R.id.internal_exams_chip_group)
internalEmptyText = findViewById(R.id.internal_empty_text)
practicalExamsCard = findViewById(R.id.practical_exams_card)
practicalExamsChipGroup = findViewById(R.id.practical_exams_chip_group)
practicalEmptyText = findViewById(R.id.practical_empty_text)
tvPracticalOptional = findViewById(R.id.tv_practical_optional)
generateButton = findViewById(R.id.generate_button)
loadingProgress = findViewById(R.id.loading_progress)
selectionStatusText = findViewById(R.id.selection_status_text)
val sems = arrayOf("Select Semester") + (1..8).map { "Semester $it" }
semesterSpinner.adapter =
ArrayAdapter(this, android.R.layout.simple_spinner_dropdown_item, sems)
writtenExamsCard.visibility = View.GONE
internalExamsCard.visibility = View.GONE
```

```
practicalExamsCard.visibility = View.GONE
generateButton.isEnabled = false
loadExamsButton.setOnClickListener { loadPublishedExams() }
generateButton.setOnClickListener { validateAndGenerate() }}
private fun setupToolbar() {
setSupportActionBar(toolbar)
supportActionBar?.title = "Generate Result"
supportActionBar?.setDisplayHomeAsUpEnabled(true)
toolbar.setNavigationOnClickListener { finish() }}
private fun loadDepartments() {
showLoading(true)
firestore.collection("departments")
.whereEqualTo("isActive", true)
.get()
.addOnSuccessListener { docs ->
showLoading(false)
departments.clear()
departments.add(DeptItem("", "Select Department"))
docs.forEach {
departments.add(DeptItem(it.id, it.getString("name") ?: it.id))}
departmentSpinner.adapter = ArrayAdapter(
this,
android.R.layout.simple_spinner_dropdown_item,
departments.map { it.name })
departmentSpinner.onItemSelectedListener =
object : AdapterView.OnItemSelectedListener {
override fun onNothingSelected(p: AdapterView<*>?) {}
override fun onItemSelected(
p: AdapterView<*>?, v: View?, pos: Int, id: Long) {
selectedDeptId = departments[pos].id
if (selectedDeptId.isNotEmpty()) loadCourses(selectedDeptId)
```

```
else resetAll()}}}
.addOnFailureListener { showLoading(false) }}
private fun loadCourses(deptId: String) {
firestore.collection("courses")
.whereEqualTo("departmentId", deptId)
.whereEqualTo("isActive", true)
.get()
.addOnSuccessListener { docs ->
courses.clear()
courses.add(CourseItem("", "Select Course"))
docs.forEach {
courses.add(CourseItem(it.id, it.getString("name") ?: it.id))}
courseSpinner.adapter = ArrayAdapter(
this,
android.R.layout.simple_spinner_dropdown_item,
courses.map { it.name })
courseSpinner.onItemSelectedListener =
object : AdapterView.OnItemSelectedListener {
override fun onNothingSelected(p: AdapterView<*>?) {}
override fun onItemSelected(
p: AdapterView<*>?, v: View?, pos: Int, id: Long) {
selectedCourseId = courses[pos].id
resetExamCards(}}}}
private fun loadPublishedExams() {
selectedSemester = semesterSpinner.selectedItemPosition
if (selectedDeptId.isEmpty()) return toast("Select a department")
if (selectedCourseId.isEmpty()) return toast("Select a course")
if (selectedSemester == 0) return toast("Select a semester")
showLoading(true)
resetExamCards()
publishedExams.clear()
```

```
firestore.collection("exams")
.whereEqualTo("marksPublished", true)
.whereEqualTo("semester", selectedSemester)
.get()
.addOnSuccessListener { docs ->
showLoading(false)
val written = mutableListOf<ExamItem>()
val internal = mutableListOf<ExamItem>()
val practical = mutableListOf<ExamItem>()
docs.forEach { doc ->
val exam = buildExamFromRaw(doc.id, doc.data ?: return@forEach)
val matchesCourse =
exam.subjects.any { it.courseId == selectedCourseId } ||
exam.courseId == selectedCourseId ||
exam.courses.contains(selectedCourseId)
if (!matchesCourse) return@forEach
val item = ExamItem(doc.id, exam.examName, exam.examType, exam)
publishedExams.add(item)
when (exam.examType.lowercase()) {
"practical" -> practical.add(item)
"internal" -> internal.add(item)
else -> written.add(item) }}
writtenExamsCard.visibility = View.VISIBLE
internalExamsCard.visibility = View.VISIBLE
practicalExamsCard.visibility = View.VISIBLE
populateChips(
written,
writtenExamsChipGroup,
selectedWrittenExamIds,
writtenEmptyText,
false)
```

```
populateChips(
internal,
internalExamsChipGroup,
selectedInternalExamIds,
internalEmptyText,
false)
populateChips(
practical,
practicalExamsChipGroup,
selectedPracticalExamIds,
practicalEmptyText,
true)
updateGenerateButton()}
.addOnFailureListener {
showLoading(false)
toast("Failed to load exams")}}
private fun populateChips(
items: List<ExamItem>,
group: ChipGroup,
selected: MutableSet<String>,
emptyText: TextView,
optional: Boolean
) { group.removeAllViews()
if (optional) {
tvPracticalOptional.text =
if (items.isEmpty())
"Optional — no practical exams found"
else
"Optional — select practical exams"}
if (items.isEmpty()) {
emptyText.visibility = View.VISIBLE
```

```
return}
emptyText.visibility = View.GONE
items.forEach { item ->
val chip = Chip(this).apply {
text = item.name
isCheckable = true
setOnCheckedChangeListener { _, checked ->
if (checked) selected.add(item.id) else selected.remove(item.id)
updateGenerateButton()}}
group.addView(chip) }}
private fun updateGenerateButton() {
val hasMinimum =
selectedWrittenExamIds.isNotEmpty() || selectedInternalExamIds.isNotEmpty()
generateButton.isEnabled = hasMinimum
val parts = mutableListOf<String>()
if (selectedWrittenExamIds.isNotEmpty()) parts.add("${selectedWrittenExamIds.size}
written")
if (selectedInternalExamIds.isNotEmpty()) parts.add("${selectedInternalExamIds.size}
internal")
if (selectedPracticalExamIds.isNotEmpty()) parts.add("${selectedPracticalExamIds.size}
practical")
selectionStatusText.text =
if (parts.isEmpty())
"Select at least one written or internal exam"
else
"${parts.joinToString(" + ")} exam(s) selected"}
private fun validateAndGenerate() {
if (selectedWrittenExamIds.isEmpty() && selectedInternalExamIds.isEmpty()) {
toast("Select at least one written or internal exam")
return}
startActivity(Intent(this, ResultDisplayActivity::class.java).apply {
```

```
putExtra("DEPT_ID", selectedDeptId)
putExtra("COURSE_ID", selectedCourseId)
putExtra("SEMESTER", selectedSemester)
putExtra("WRITTEN_EXAM_IDS", selectedWrittenExamIds.toTypedArray())
putExtra("INTERNAL_EXAM_IDS", selectedInternalExamIds.toTypedArray())
putExtra("PRACTICAL_EXAM_IDS", selectedPracticalExamIds.toTypedArray())})}
private fun resetAll() {
courses.clear()
courseSpinner.adapter =
ArrayAdapter(this, android.R.layout.simple_spinner_dropdown_item, listOf("Select
Course"))
resetExamCards()}
private fun resetExamCards() {
listOf(writtenExamsChipGroup,
internalExamsChipGroup,
practicalExamsChipGroup).forEach { it.removeAllViews() }
selectedWrittenExamIds.clear()
selectedInternalExamIds.clear()
selectedPracticalExamIds.clear()
writtenExamsCard.visibility = View.GONE
internalExamsCard.visibility = View.GONE
practicalExamsCard.visibility = View.GONE
generateButton.isEnabled = false
selectionStatusText.text = "Select at least one written or internal exam"}
private fun showLoading(show: Boolean) {loadingProgress.visibility = if (show)
View.VISIBLE else View.GONE}
private fun toast(msg: String) =
Toast.makeText(this, msg, Toast.LENGTH_SHORT).show()
private fun tsLong(v: Any?): Long =
when (v) {
is Timestamp -> v.toDate().time
```

```
is Long -> v
is Double -> v.toLong()
is Int -> v.toLong()
else -> 0L}
@Suppress("UNCHECKED_CAST")
private fun buildExamFromRaw(id: String, raw: Map<String, Any>): Exam {
val subjects = mutableListOf<ExamSubject>()
(raw["subjects"] as? List<*>)?.forEach {
val s = it as? Map<*, *> ?: return@forEach
subjects.add(
ExamSubject(
subjectId = s["subjectId"] as? String ?: "",
subjectName = s["subjectName"] as? String ?: "",
subjectCode = s["subjectCode"] as? String ?: "",
courseId = s["courseId"] as? String ?: "",
courseName = s["courseName"] as? String ?: "",
examDate = tsLong(s["examDate"]),
examType = s["examType"] as? String ?: "Written",
maxMarks = tsLong(s["maxMarks"]).toInt(),
writtenMaxMarks = tsLong(s["writtenMaxMarks"]).toInt(),
internalMaxMarks = tsLong(s["internalMaxMarks"]).toInt(),
assignedFacultyId = s["assignedFacultyId"] as? String ?: ""))}
val statusMap = mutableMapOf<String, SubjectMarksStatus>()
(raw["subjectsMarksStatus"] as? Map<*, *>)?.forEach { (k, v) ->
val m = v as? Map<*, *> ?: return@forEach
statusMap[k.toString()] = SubjectMarksStatus(
subjectId = m["subjectId"] as? String ?: "",
marksSubmitted = m["marksSubmitted"] as? Boolean ?: false,
moderated = m["moderated"] as? Boolean ?: false,
moderatedAt = tsLong(m["moderatedAt"]))}
return Exam(
```

```
id = id,
examName = raw["examName"] as? String ?: "",
semester = tsLong(raw["semester"]).toInt(),
academicYear = raw["academicYear"] as? String ?: "",
startDate = tsLong(raw["startDate"]),
examType = raw["examType"] as? String ?: "",
courseId = raw["courseId"] as? String ?: "",
courseName = raw["courseName"] as? String ?: "",
isActive = raw["isActive"] as? Boolean ?: true,
courses = (raw["courses"] as? List<*>)?.filterIsInstance<String>() ?: emptyList(),
subjects = subjects,
marksEntryEnabled = raw["marksEntryEnabled"] as? Boolean ?: false,
marksPublished = raw["marksPublished"] as? Boolean ?: false,
subjectsMarksStatus = statusMap)}
data class DeptItem(val id: String, val name: String)
data class CourseItem(val id: String, val name: String)
data class ExamItem(val id: String, val name: String, val type: String, val exam: Exam)}
```

4.2 Testing Approach

4.2.1 Functional and Integration Testing

Functional and integration testing of SYNAPSE focused on validating all key modules, including role-based login, dashboard navigation, attendance management, examination scheduling, and result generation. Each feature was tested for correct operation individually and in combination, ensuring smooth interaction with Firebase Authentication and Firestore. The testing confirmed reliable data synchronization, secure access control, and consistent performance across different user roles under realistic usage scenarios.

TEST CASE ID	TEST CASE DESCRIPTION	OBJECTIVE	EXPECTED RESULT	ACTUAL RESULT	REMARK
TC-01	Login with valid email and password	Verify correct authentication	User is logged in and redirected to their role-specific dashboard	As expected	**Pass**
TC-02	Login with invalid email format	Validate input checking	Error message displayed requesting valid email	As expected	**Pass**
TC-03	Login with incorrect password	Verify authentication failure handling	Login fails and error message shown	As expected	**Pass**
TC-04	Empty login fields	Check input validation	User cannot proceed; validation errors shown	As expected	**Pass**
TC-05	Role-based access	Ensure user sees only authorized modules	Student redirected to Student Dashboard, Faculty to Faculty Dashboard, HOD to HOD Dashboard, COE to COE Dashboard	As expected	**Pass**
TC-06	Attendance entry	Validate data entry and saving	Attendance saved correctly in Firestore	As expected	**Pass**
TC-07	Duplicate attendance entry	Check prevention of duplicate data	System prevents saving duplicate record	As expected	**Pass**
TC-08	Exam selection	Verify correct exam retrieval	Relevant exams for selected department, course, and semester are displayed	As expected	**Pass**
TC-09	Result generation	Ensure only valid selections can generate results	Generate button enabled only when exams selected; result screen shows correct data	As expected	**Pass**
TC-10	Logout	Confirm session termination	User is logged out and redirected to login screen	As expected	**Pass**
TC-11	Network failure handling	Test resilience to network issues	App displays error gracefully without crashing	As expected	**Pass**
TC-12	Unauthorized role access	Test security and access control	Access denied and user cannot enter restricted modules	As expected	**Pass**

CHAPTER 5

RESULTS

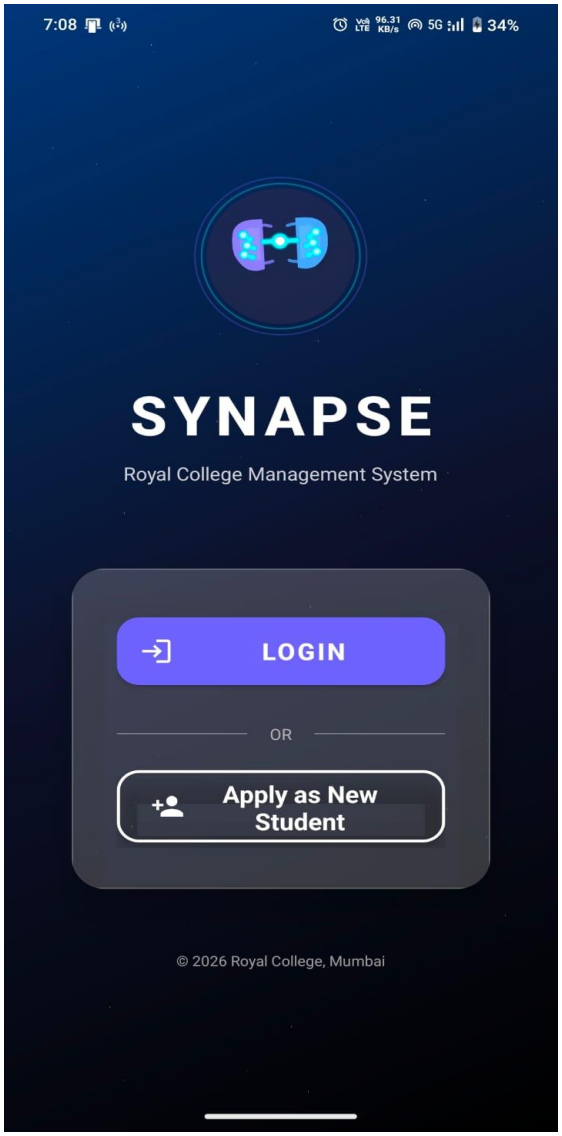


Fig 5.1 Login/Register

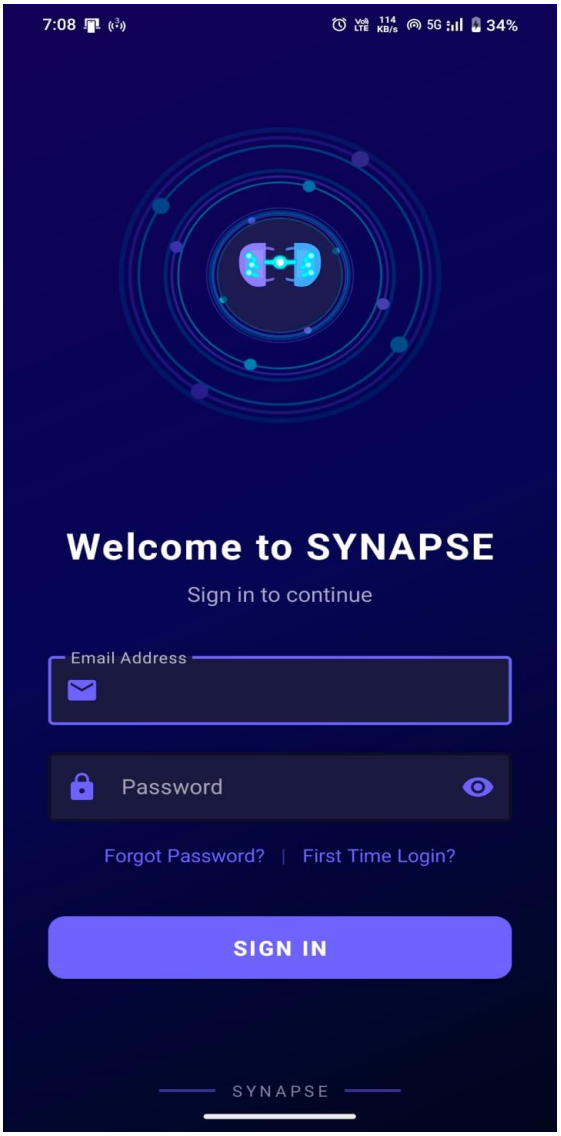


Fig 5.2 Login Screen

Dashboards

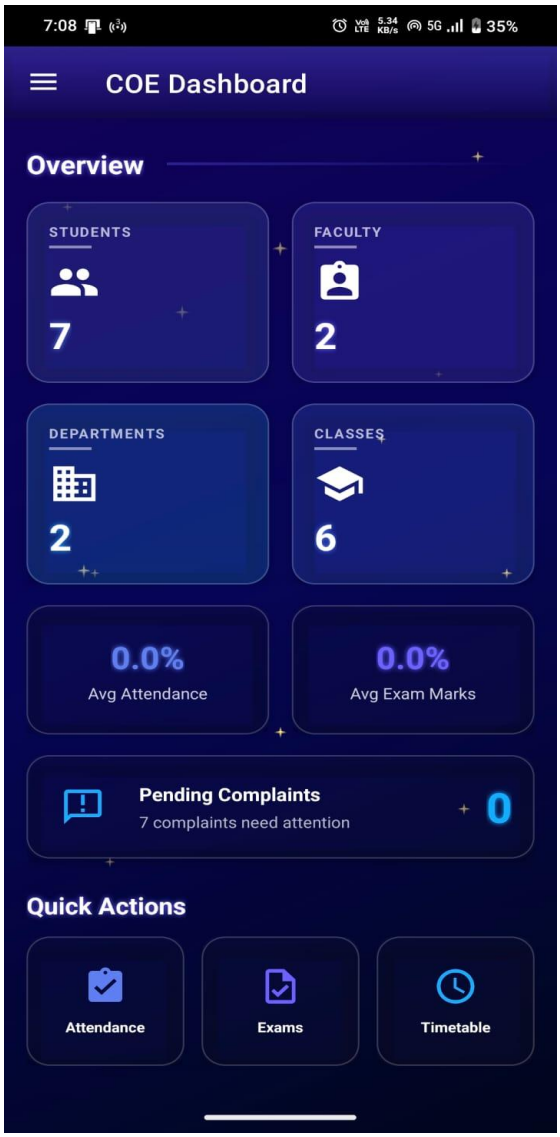


Fig 5.3 COE Dashboard

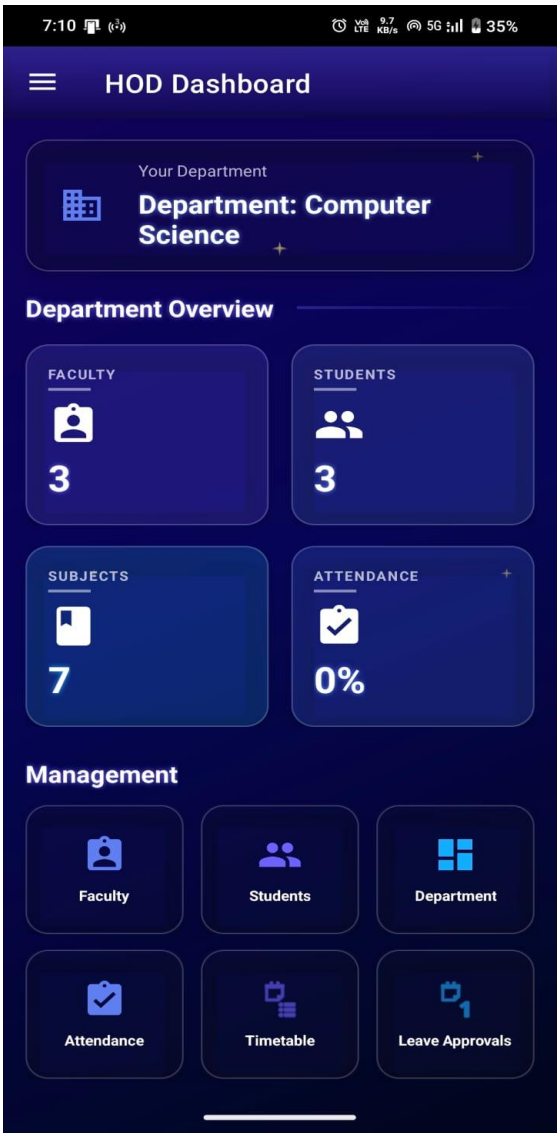


Fig 5.4 HOD Dashboard

Dashboards

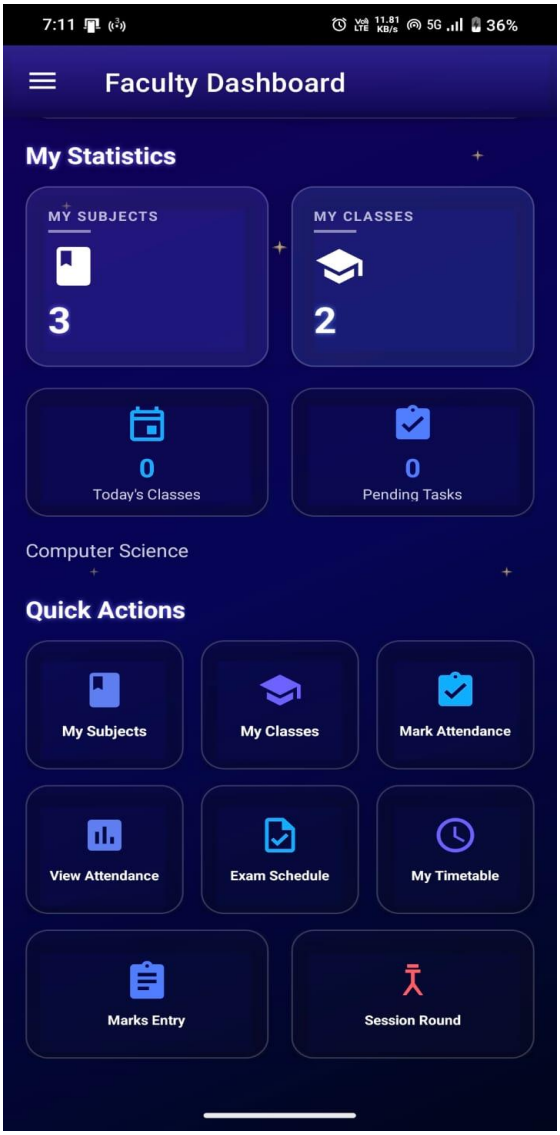


Fig 5.5 Faculty Dashboard

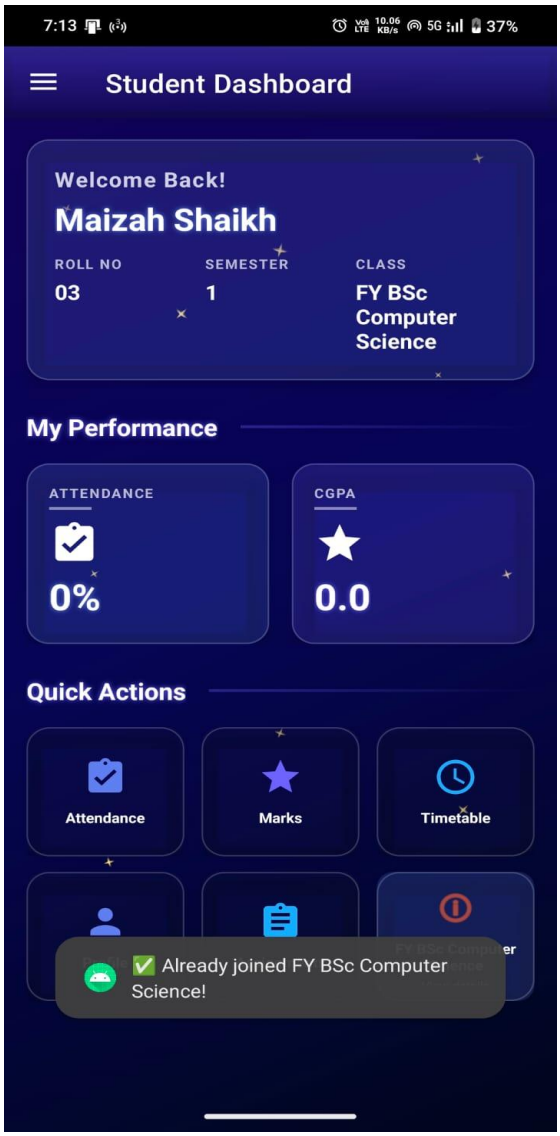


Fig 5.6 Student Dashboard

COE Activities

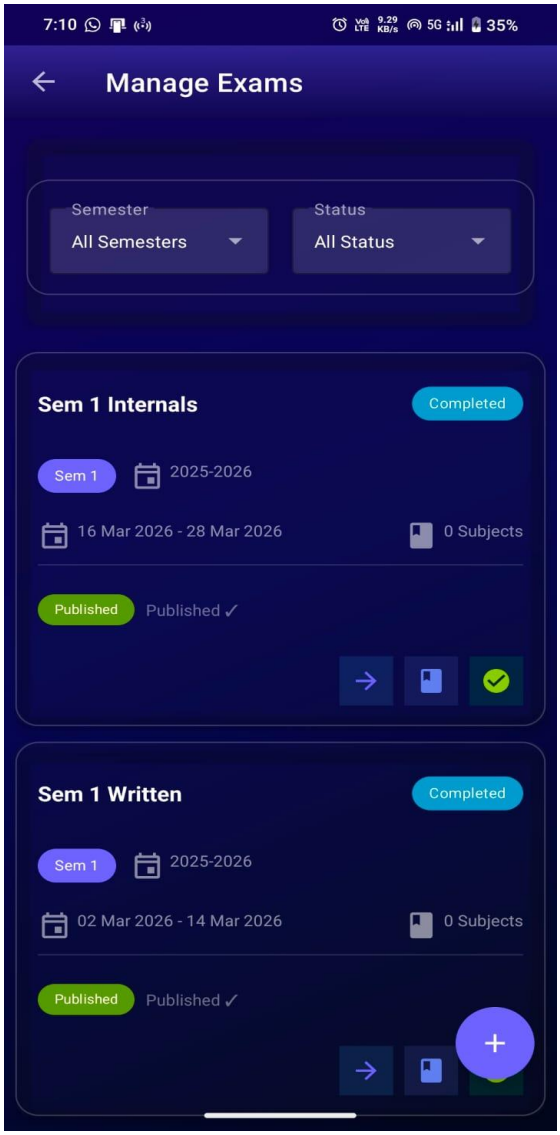


Fig 5.7 Manage Exams

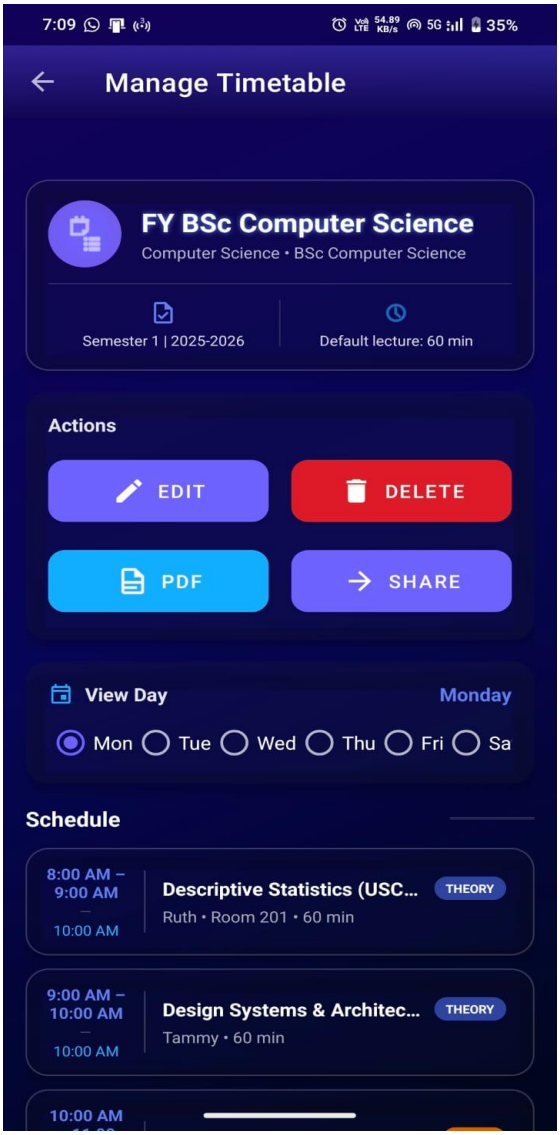


Fig 5.8 Schedule Management

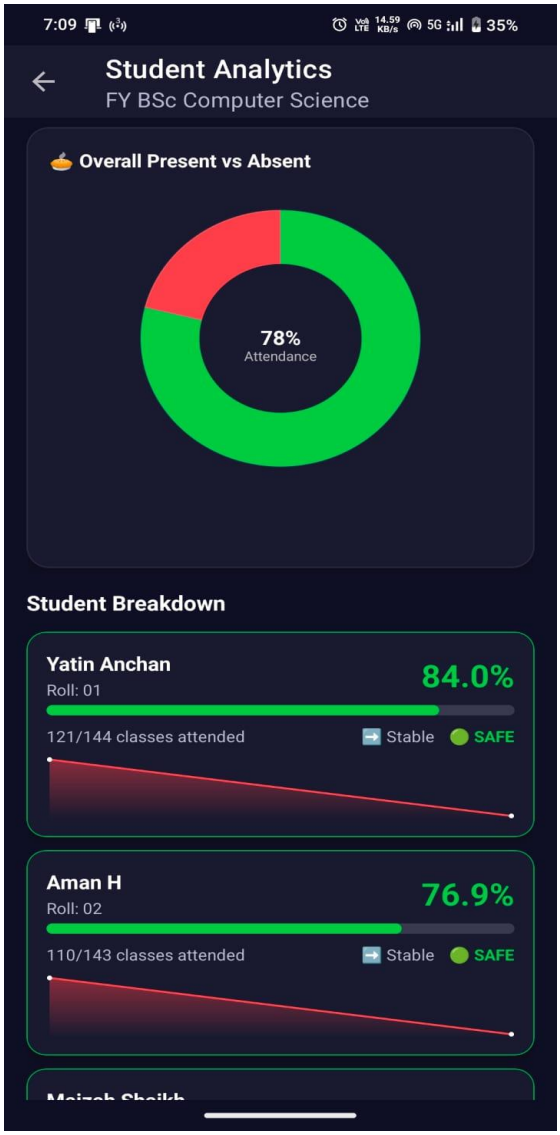


Fig 5.9 Attendance Analytics (Student-wise)

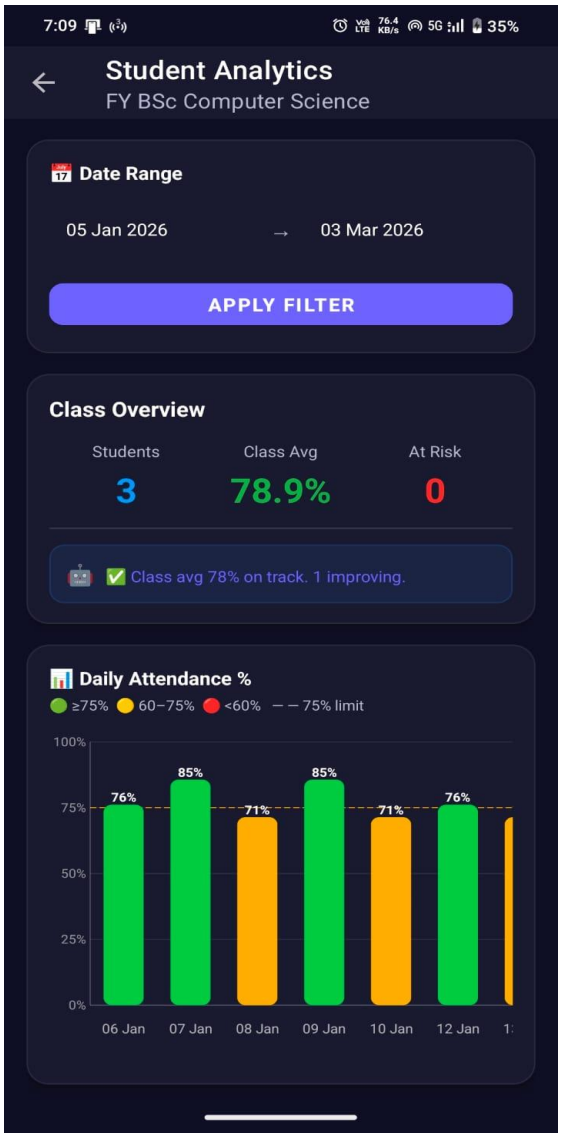


Fig 5.10 Attendance Analytics (Date-wise)

HOD Activities

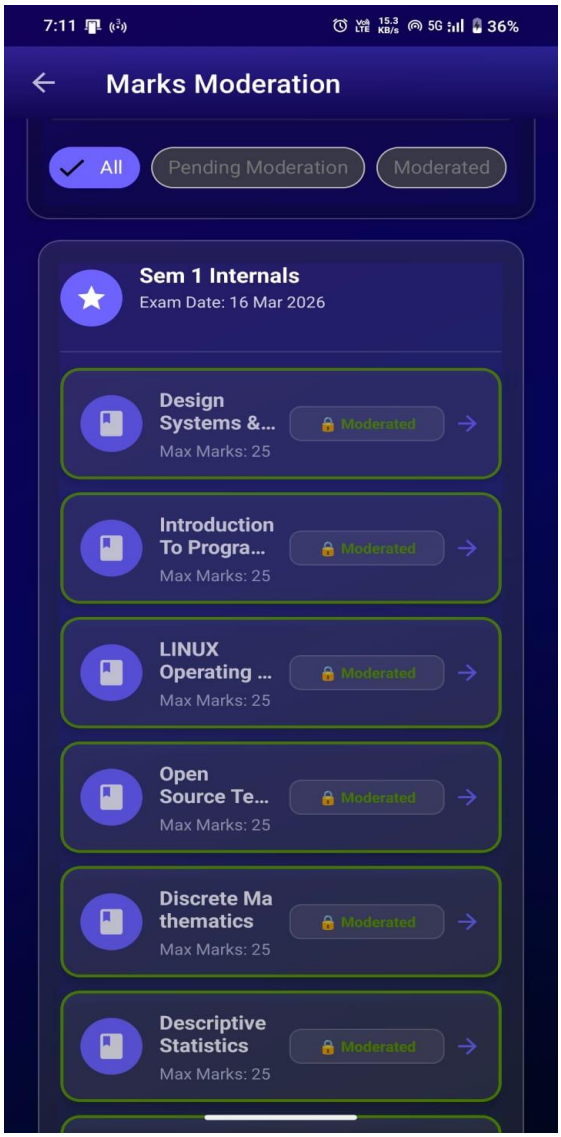


Fig 5.11 Exam Moderation List

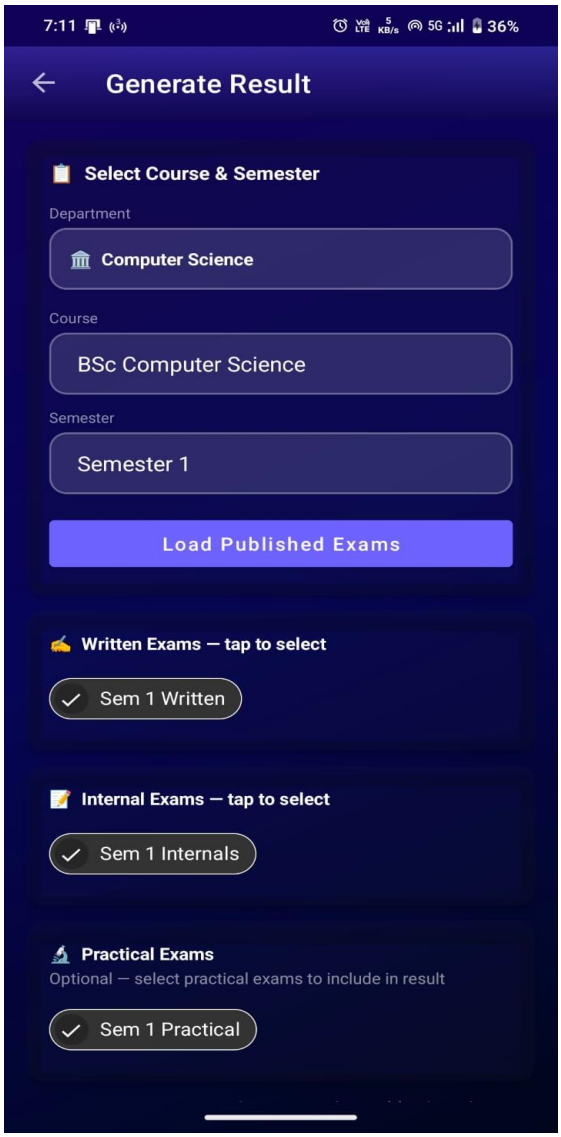


Fig 5.12 Result Gazette PDF Generation

Faculty Activities

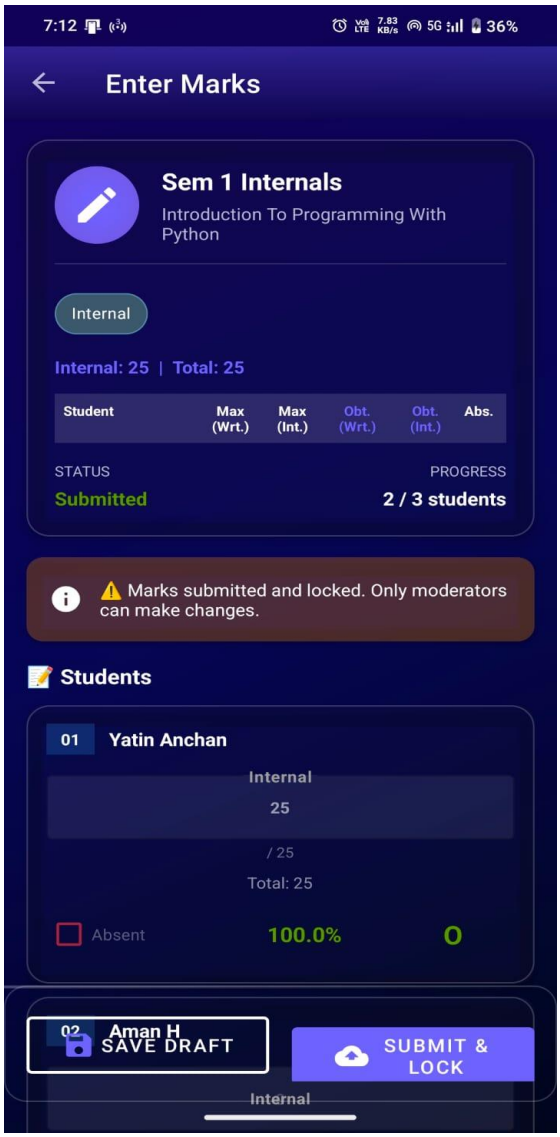


Fig 5.13 Marks Entry

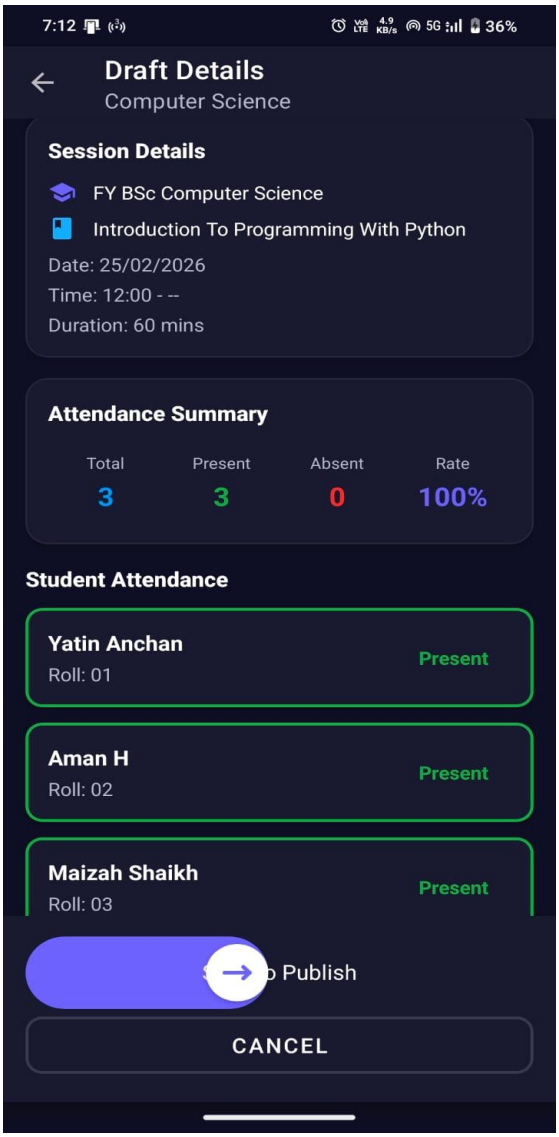


Fig 5.14 Attendance Sumission

PDF Generated

26/02/2026

ROYAL COLLEGE OF ARTS, SCIENCE AND COMMERCE
(AUTONOMOUS)
BSc Computer Science
SEM 1 INTERNALS
March 2026
TIME TABLE

Time: 09:00 AM to 09:30 AM

Day & Date	Subject Code & Name	Ref
Monday 16/03/2026	USCS101 Design Systems & Architecture	1
Tuesday 17/03/2026	USCS102 Introduction To Programming With Python	2
Friday 20/03/2026	USCS103 LINUX Operating System	3
Monday 23/03/2026	USCS104 Open Source Technology	4
Tuesday 24/03/2026	USCS105 Discrete Mathematics	5
Thursday 26/03/2026	USCS106 Descriptive Statistics	6
Saturday 28/03/2026	USCS107 Soft skills	7

EXAMINATION COMMITTEE Controller of Examination Principal

Fig 5.15 Exam Timetable PDF

ROYAL COLLEGE OF ARTS, SCIENCE AND COMMERCE, MIRA ROAD (EAST)
Department of COMPUTER SCIENCE
BSC COMPUTER SCIENCE
Semester 1 — Examination Results

Roll No: 01 **Name: Yatin Anchan**

CODE	SUBJECT	WRT	INT	PRC	TOTAL	%	GR	GP
	Soft skills	65/75	15/25	–	80/100	80.0	A+	9.0
	Descriptive Statistics	35/75	13/25	11/15	59/115	51.3	C	5.0
	Design Systems & Architecture	52/75	22/25	15/15	89/115	77.4	A	8.0
	Introduction To Programming With Python	60/75	25/25	11/15	96/115	83.5	A+	9.0
	LINUX Operating System	62/75	25/25	14/15	101/115	87.8	A+	9.0
	Discrete Mathematics	55/75	17/25	10/15	82/115	71.3	A	8.0
	Open Source Technology	72/75	20/25	13/15	105/115	91.3	O	10.0

Grand Total: 612 / 790 **Percentage: 77.47%** **CGPI: 8.29 / 10**
Grade: A | Result: FIRST CLASS

Fig 5.16 Result Gazette PDF

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

The SYNAPSE system successfully integrates key college management functionalities, including attendance tracking, examination scheduling, course and semester management, and result generation, into a single, user-friendly mobile platform. Its role-based access control ensures that administrators, faculty, HODs, and students have access only to relevant modules, maintaining security, privacy, and accountability. By processing core operations locally on the device, the system delivers fast response times while reducing dependence on constant network connectivity, allowing uninterrupted academic operations even in low-bandwidth environments.

Integration with Firebase services provides secure authentication, reliable data storage, and real-time synchronization of academic records across devices. This hybrid approach of local processing combined with cloud support balances performance, scalability, and privacy. Functional and integration testing verified that all modules function correctly, interact seamlessly, and provide consistent and accurate outputs for different user roles under realistic usage scenarios. Moreover, the system demonstrates strong error handling, intuitive navigation, and responsive design, ensuring that users with varying technical proficiency can efficiently utilize the platform.

Overall, SYNAPSE offers a modern, scalable, and efficient solution for institutional management. It reduces manual effort, minimizes errors, and streamlines workflows, making academic and administrative operations more coordinated and transparent. The system lays a strong foundation for future enhancements, including AI-driven insights, automated reporting, predictive analytics, and integration with third-party educational tools. With its robust architecture and forward-looking design, SYNAPSE positions itself as a dependable, comprehensive, and future-ready platform that can evolve with the growing needs of academic institutions, supporting better decision-making, improved student outcomes, and overall institutional excellence.

6.2 Future Work

While the current SYNAPSE system provides a robust and comprehensive solution for college management, there are several opportunities to enhance its functionality, intelligence, and user experience. Future developments can focus on integrating advanced technologies, improving automation, and expanding the platform to support emerging needs of academic institutions. These enhancements aim to make SYNAPSE not only a management system but also a proactive assistant that can guide, inform, and streamline operations dynamically.

Potential Future Work:

- **AI-Based Role Assistant:** Implement an AI assistant that adapts to each user's role (student, faculty, HOD, COE) to provide contextual guidance, reminders, and suggestions.
- **Event Management Enhancements:** Enable automated event creation, scheduling, and RSVP tracking, including notifications and guest management.
- **Advanced Attendance Logging:** Integrate features such as facial recognition, NFC/QR-based digital attendance, and analytics dashboards for trends and defaulters.
- **Digital Student and Staff IDs:** Introduce secure digital ID cards for access control, verification, and campus services.
- **Intelligent Notifications:** Push customized alerts for exams, deadlines, approvals, and institutional announcements based on role and priorities.
- **Enhanced Reporting and Analytics:** Develop AI-powered insights for student performance, faculty effectiveness, attendance patterns, and institutional KPIs.
- **Integration with Learning Tools:** Connect SYNAPSE with online learning platforms, digital libraries, and course management tools for a unified academic ecosystem.
- **Mobile-First Enhancements:** Improve offline functionality, real-time synchronization, and interactive dashboards for tablets and smartphones.

CHAPTER 7

REFERENCES AND PLAGIARISM REPORT

References

[1] R. Laudon and J. Laudon, *Management Information Systems: Managing the Digital Firm*, 16th ed., Pearson Education, 2020.

[2] K. C. Laudon, "Essentials of management information systems," Pearson, New Delhi, India, 2019.

[3] Royal College of Arts, Science and Commerce, Mira Road (E), "Official College Website," Mumbai, India. [Online]. Available: https://www.royalcollege.edu.in

[4] University of Mumbai, "Faculty of Science – Computer Science Syllabus," Mumbai, India. [Online].

[5] Maharashtra State Board of Secondary and Higher Secondary Education, "SSC Examination Information," Pune, India. [Online].

[6] Maharashtra State Board of Secondary and Higher Secondary Education, "HSC Examination Information," Pune, India. [Online].

[7] A. Dennis, B. Wixom, and D. Tegarden, *Systems Analysis and Design: An Object-Oriented Approach with UML*, 5th ed., Wiley India, 2018.

[8] T. Connolly and C. Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th ed., Pearson Education, 2017.

[9] J. W. Satzinger, R. B. Jackson, and S. D. Burd, *Systems Analysis and Design in a Changing World*, Cengage Learning, 2016.

[10] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed., Addison-Wesley, 2018.

[11] Oracle Corporation, "Role-Based Access Control (RBAC) Concepts," Oracle Documentation. [Online].

[12] IBM Corporation, "Management Information Systems Overview," IBM Documentation. [Online].

Plagiarism Report

Dupli Checker

Date: 03-03-2026

Plagiarism Scan Report

0% Plagiarism	0% Exact Match
	0% Partial Match

100% Unique

Words	388
Characters	3043
Sentences	19
Paragraphs	10
Read Time	2 minute(s)
Speak Time	3 minute(s)

Content Checked For Plagiarism

1. Introduction

In today's rapidly evolving educational environment, colleges and academic institutions require efficient, centralized and secure systems to manage academic and administrative operations. Traditional manual processes and fragmented digital tools often lead to inefficiencies, data redundancy, lack of transparency and increased administrative workload. To address these challenges, Management Information Systems (MIS) play a vital role in streamlining institutional workflows and enhancing decision-making processes.

SYNAPSE is a comprehensive and ultra-niche Management and Information System (MIS) designed specifically for colleges to support institutional management, faculty coordination and administrative staff operations. The system aims to provide a unified digital platform that integrates academic, examination, departmental and event management functionalities under a single secure framework.

SYNAPSE follows a role-based access control model, ensuring that users such as System Administrators, Principals, Vice Principals (Coordinators of Examination), Heads of Departments (HODs), Faculty Members and Committee Heads can access features relevant to their responsibilities. This structured access enhances data security, accountability and operational efficiency.

The system facilitates end-to-end academic processes including course, semester and class creation, examination scheduling and result generation. Additional modules such as attendance management, department and committee management and event planning with RSVP and guest tracking further extend the system's capabilities. By centralizing these operations, SYNAPSE minimizes manual effort, reduces errors and improves institutional coordination.

Overall, SYNAPSE is designed to modernize college administration by offering a scalable, secure and efficient MIS solution tailored to the specific needs of academic institutions.

1.1 Background and Project Overview

In the modern academic landscape, colleges face mounting pressures from growing student enrollments, complex administrative demands and the need for real-time data access. Manual record-keeping and scattered software solutions have long plagued institutions, causing delays in processes like exam scheduling, attendance tracking and event coordination. These inefficiencies not only burden staff but also hinder faculty collaboration and institutional decision-making.

SYNAPSE emerges as a tailored response to these pain points, building on established MIS principles to create a cohesive platform for college operations. It centralizes key functions ranging form academic structure management to secure role-based access into an intuitive system that adapts to diverse user needs, such as those of Principals, HODs and faculties.

This project overview positions SYNAPSE as more than just software; it's a strategic tool that transforms fragmented workflows into streamlined, data-driven processes, paving the way for enhanced transparency and scalability in educational administration.

Dupli Checker Date: 03-03-2026

Plagiarism Scan Report

0% Plagiarism	0% Exact Match
	0% Partial Match

100% Unique

Words	532
Characters	4080
Sentences	42
Paragraphs	27
Read Time	3 minute(s)
Speak Time	4 minute(s)

Content Checked For Plagiarism

1.2 Objectives

The primary objectives of SYNAPSE MIS guide its development as a practical solution for college administration challenges. These goals ensure the system delivers efficiency, security and scalability tailored to academic institutions.

1. Design a centralized platform to manage academic and administrative tasks, reducing manual efforts and data mismanagement across departments.
2. Implement role-based access control for users like admins, principals, HODs, faculty and committee heads, ensuring secure and relevant feature access.
3. Automate core academic processes, including course, semester, class creation, faculty assignments and structured record-keeping.
4. Streamline end-to-end examination management, from scheduling and assessments to result generation and reporting.
5. Enable digital attendance tracking for students and faculty, with centralized storage accessible to authorized personnel.
6. Support department and committee oversight, including member assignments, roles and activity logging for better coordination.
7. Enhance transparency and decision-making through real-time data access and organized reporting tools.
8. Build a modular, scalable architecture ready for future additions like analytics, mobile support or integrations.
9. Prioritize user-friendly design and affordability, making advanced MIS features viable for typical college setups.

These objectives directly address inefficiencies in traditional systems, positioning SYNAPSE as a transformative tool for modern education management.

1.3 Purpose and Scope

1.3.1 Purpose

SYNAPSE is developed with a focused purpose to deliver an affordable, secure and centralized Management Information System (MIS) that revolutionizes college administration by tackling inefficiencies in manual and fragmented process and data handling. In today's academic world, institution struggle with outdated record-keeping, disjointed tools and limited access to real-time data, leading to errors and delay and poor coordination.

The system detects and resolves these operational gaps by providing role-based access, automating academic workflows like course management and exams also enabling seamless handling of attendance, departments, classes and students. Upon streamlining these core functions, SYNAPSE generates structured data, enhances, transparency and notifies authorized users instantly preventing administrative bottlenecks

before they escalate.
By removing the need for multiple disparate systems and prioritizing intuitive design, SYNAPSE makes advanced institutional management accessible to colleges of all sizes, fostering efficiency, accountability and smarter decision-making across the board.

1.3.2 Scope

SYNAPSE MIS is a native Android application tailored for college administration, providing a lightweight, mobile-first platform that works seamlessly on standard Android devices without needing extra hardware or web dependencies. It's designed to be intuitive and accessible for users like admins, principals, HODs, faculties, running directly on their phones or tablets to handle daily academic and operational tasks efficiently.
The app launches core modules upon login with role-based access, continuously managing workflows like course creation and exam scheduling. It monitors activities such as attendance marking, department updates in real-time, logging data locally and alerting relevant users to issues before they escalate.
Key features within scope include:

- Role-based access control tailored for Android, with secure profiles for Principal, HOD, Faculty and others.
- Academic tools for building courses, semesters, classes and faculty assignments on the go.
- Full examination cycle: scheduling, assessments and result processing.
- Touch-based attendance entry with synced, centralized views.
- Department and class management, including member assignments.

This Android-centric scope targets essential college functions, skipping complex financials or external links for simplicity, while building in modularity for future enhancements like cloud sync.

Matched Source

Congratulations !

No Plagiarism Found

Check By: Dupli Checker

T.Y.B. Sc. Computer Science

Semester VI

A.Y. 2025-2026

Accepted Project Proposal

On

SYNAPSE (MIS for Institutions & Faculties)

Yatin Anil Anchan

Roll No: 23

Introduction

In today's rapidly evolving educational environment, colleges and academic institutions require efficient, centralized, and secure systems to manage academic and administrative operations. Traditional manual processes and fragmented digital tools often lead to inefficiencies, data redundancy, lack of transparency, and increased administrative workload. To address these challenges, Management Information Systems (MIS) play a vital role in streamlining institutional workflows and enhancing decision-making processes.

Synapse is a comprehensive and ultra-niche **Management and Information System (MIS)** designed specifically for colleges to support institutional management, faculty coordination, and administrative staff operations. The system aims to provide a unified digital platform that integrates academic, examination, departmental, and event management functionalities under a single secure framework.

Synapse follows a **role-based access control model**, ensuring that users such as System Administrators, Principals, Vice Principals (Coordinators of Examination), Heads of Departments (HODs), Faculty Members, and Committee Heads can access features relevant to their responsibilities. This structured access enhances data security, accountability, and operational efficiency.

The system facilitates end-to-end academic processes including **course, semester, and class creation**, examination scheduling, and **result generation**. Additional modules such as **attendance management**, **department and committee management**, and **event planning with RSVP and guest tracking** further extend the system's capabilities. By centralizing these operations, Synapse minimizes manual effort, reduces errors, and improves institutional coordination.

Overall, Synapse is designed to modernize college administration by offering a scalable, secure, and efficient MIS solution tailored to the specific needs of academic institutions.

Objectives

The main objectives of the **Synapse – Management and Information System (MIS)** are as follows:

1. To design and develop a centralized MIS for colleges that efficiently manages academic and administrative activities.
2. To implement a **role-based access control system** that ensures secure and authorized access for different users such as administrators, principals, examination coordinators, HODs, faculty members, and committee heads.
3. To automate academic processes including **course, semester, and class creation**, thereby reducing manual workload and data redundancy.
4. To provide an end-to-end **examination management system**, covering exam scheduling, assessment processes, and result generation.
5. To facilitate effective **attendance management** for students and faculty (if implemented).
6. To manage **departments and institutional committees** through structured digital modules.
7. To support **event management**, including event creation, RSVP handling, and guest tracking.
8. To improve data accuracy, transparency, and accessibility across the institution.
9. To enhance decision-making capabilities of institutional authorities through organized and real-time information.
10. To develop a scalable and modular system that can be extended with additional features in the future.

Scope

The scope of the **Synapse – Management and Information System (MIS)** is focused on providing a centralized digital platform for managing academic and administrative operations of a college. The system is designed to be used by institutional authorities, faculty members, and administrative staff to streamline workflows and improve operational efficiency.

Synapse includes **role-based access** for various users such as System Administrators, Principals, Vice Principals (Coordinator of Examinations), Heads of Departments, Faculty Members, and Committee Heads. Each role is granted controlled access to relevant modules, ensuring data security and proper authorization.

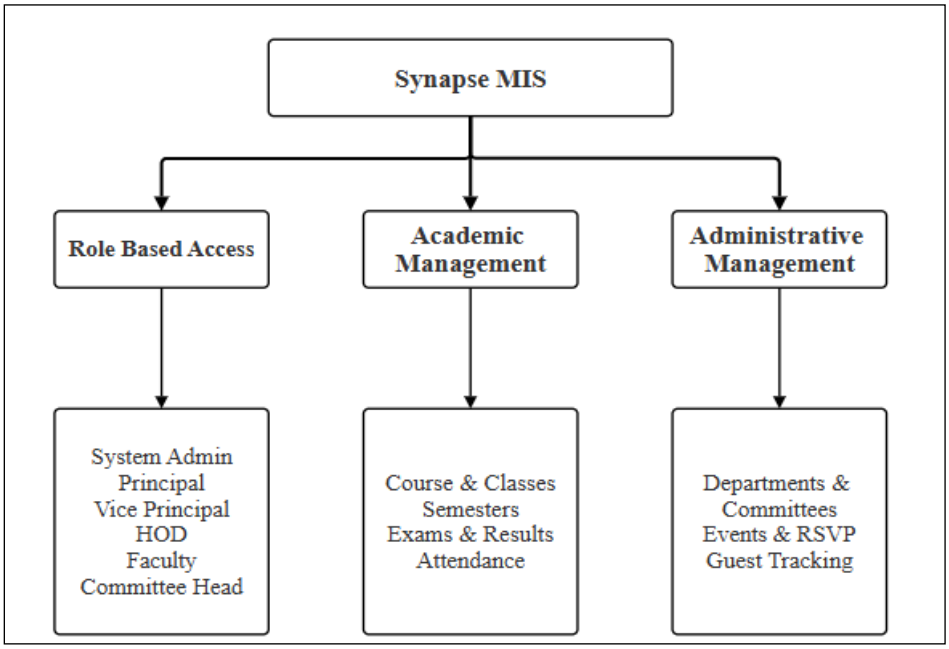
The system supports core academic functionalities including **course, semester, and class creation**, along with **examination management** from scheduling to result generation. It also covers **department and committee management**, enabling structured handling of institutional units and responsibilities.

Additional features within the scope include **event management**, allowing institutions to organize events with RSVP handling and guest tracking. **Attendance management** may be included as an optional or extendable module based on institutional requirements.

The scope of Synapse is limited to college-level institutions and does not include financial management, payroll processing, or external university-level systems in its current version. However, the system is designed with scalability in mind, allowing future enhancements such as analytics, mobile integration, and advanced reporting.

Overall, Synapse aims to provide a secure, efficient, and modular MIS tailored specifically for academic institutions

Methodologies: (diagrammatic representation)



1. Development Approach

For the development of the **Synapse MIS**, the **Iterative and Incremental Development Model** was adopted. This approach allows the system to be developed in small, manageable modules, with each iteration adding new features or enhancing existing ones.

This methodology was chosen because:

- The system involves multiple independent modules.
- Continuous feedback from users (faculty and administrators) is essential.
- Features can be tested, refined, and expanded progressively.
- It supports scalability and future enhancements.

2. System Modules

The Synapse MIS is divided into the following logical modules:

1. Role-Based Access Control Module
2. Academic Management Module
3. Examination Management Module
4. Attendance Management Module
5. Department & Committee Management Module
6. Event Management with RSVP & Guest Tracking.

3. Module-wise Implementation

1. Role-Based Access Control Module

- Different user roles such as System Admin, Principal, Vice Principal (CoE), HOD, Faculty, and Committee Head are defined.
- Each role is granted specific permissions.
- Ensures data security, accountability, and controlled access.

2. Academic Management Module

- Allows creation and management of courses, semesters, and classes.
- Assigns faculty to subjects and classes.
- Maintains structured academic data.

3. Examination Management Module

- Supports exam scheduling and coordination.
- Enables result generation and storage.
- Accessible primarily to CoE and authorized authorities.

4. Attendance Management Module

- Faculty can mark and manage attendance for assigned classes.
- Attendance records are stored centrally.
- Data can be viewed by HODs and administrators.

5. Department & Committee Management Module

- Manages academic departments and institutional committees.
- Assigns committee heads and members.
- Stores roles, responsibilities, and activity records.

6. Event Management with RSVP & Guest Tracking

- Enables creation and management of institutional events.
- Supports RSVP tracking for participants.
- Maintains guest and attendance records for events.

Project Timeline & Gantt Chart

T.Y.BSc. Computer Science Semester 6		Year 2025-2026															
Project Gantt Chart	Time Requirements	November				December				January				February			
		W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4	W1	W2	W3	W4
Requirement Gathering & Feasibility Study	Estimated																
Project Planning	Estimated																
System Design	Estimated																
Implementation / Coding	Estimated																
Testing	Estimated																
Deployment Preparation	Estimated																

Expected Outcomes:

The expected outcome of the **Synapse – Management and Information System (MIS)** is the successful development of a centralized, secure, and role-based digital platform that efficiently manages academic and administrative operations of a college. The system aims to reduce manual effort, improve data accuracy, and enhance coordination among institutional stakeholders.

What the System Will Do (Minimum Viable Product – MVP)

1. **Centralized Institutional Management**
   Synapse will provide a single unified platform to manage academic, administrative, and event-related data, eliminating the need for multiple disconnected systems.
2. **Role-Based Access and Security**
   The system will enforce role-based access control, ensuring that users such as administrators, principals, examination coordinators, HODs, faculty members, and committee heads can access only authorized features and data.
3. **Academic Structure Management**
   The system will enable efficient creation and management of courses, semesters, classes, and faculty assignments, ensuring structured academic records.
4. **Examination Management**
   Synapse will support examination scheduling and result generation, providing a streamlined workflow for examination authorities and improving transparency.
5. **Attendance Management**
   Faculty members will be able to mark and manage attendance digitally, with centralized storage and controlled access for higher authorities.
6. **Department and Committee Management**
   The system will maintain organized records of departments, committees, members, and roles, improving institutional coordination and accountability.
7. **Event Management with RSVP and Guest Tracking**
   Synapse will allow institutions to create events, manage participant RSVPs, and track guest attendance, making event organization more efficient.

8. **Improved Decision-Making and Reporting**

   Authorized users will have access to structured and up-to-date information, supporting better administrative decision-making.

9. **Scalability and Future Enhancement Readiness**

   The system will be designed to allow future enhancements such as analytics dashboards, reporting tools, mobile access, and integration with other institutional systems.

References:

[1] R. Laudon and J. Laudon, *Management Information Systems: Managing the Digital Firm*, 16th ed., Pearson Education, 2020.

[2] K. C. Laudon, "Essentials of management information systems," Pearson, New Delhi, India, 2019.

[3] Royal College of Arts, Science and Commerce, Mira Road (E), "Official College Website," Mumbai, India. [Online]. Available: https://www.royalcollege.edu.in

[4] University of Mumbai, "Faculty of Science – Computer Science Syllabus," Mumbai, India. [Online].

[5] Maharashtra State Board of Secondary and Higher Secondary Education, "SSC Examination Information," Pune, India. [Online].

[6] Maharashtra State Board of Secondary and Higher Secondary Education, "HSC Examination Information," Pune, India. [Online].

[7] A. Dennis, B. Wixom, and D. Tegarden, *Systems Analysis and Design: An Object-Oriented Approach with UML*, 5th ed., Wiley India, 2018.

[8] T. Connolly and C. Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*, 6th ed., Pearson Education, 2017.

[9] J. W. Satzinger, R. B. Jackson, and S. D. Burd, *Systems Analysis and Design in a Changing World*, Cengage Learning, 2016.

[10] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*, 3rd ed., Addison-Wesley, 2018.

[11] Oracle Corporation, "Role-Based Access Control (RBAC) Concepts," Oracle Documentation. [Online].

[12] IBM Corporation, "Management Information Systems Overview," IBM Documentation. [Online].